

CSS 5202

DEEP LEARNING AND APPLICATIONS



- Self driving car
- Google Home devices:
- Amazon (Alexa) and Apple's (Siri) digital voice assistants,
- Text-to-speech synthesis
- Language identification
- English-to-French translation
- Handwritten character recognition
- Optical character recognition: identification of printed chars using photoelectric devices and computer s/w
- Video-to-textual description (Annotations to video)
- Syntactic parsing for natural language processing : Ability of computer to understand the human language.

Some more examples of applied deep learning include the following:

- Automated image sharpening: **Sharpening** a digital **image means** removing blur, enhancing details, and dehazing: degraded visibility caused by atmospheric conditions
- Automating image upscaling
- Speech reconstruction from silent video
- Image autofill for missing regions
- Automated image captioning (process of generating textual description from an **image**)



- Turning hand-drawn doodles into stylized artwork (Doodle means a rough drawing i.e. pencil sketch to photo)

AI, ML, DL, NN and DS

AI, ML, DL, NN and DS

Artificial intelligence

AI: is just a program or machine's ability to learn and think for itself, in such a way that it is capable of developing skills as it practices and gains experience. Its purpose is to support and enhance human activities.

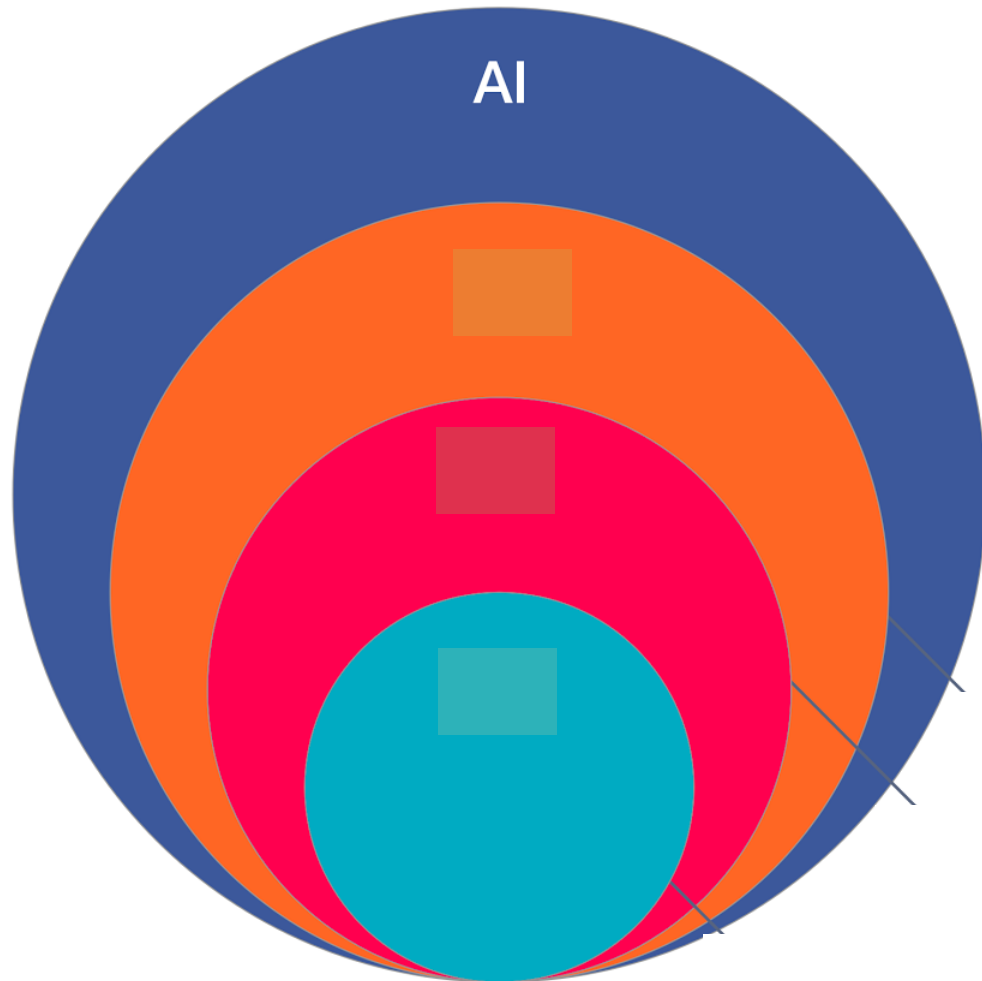
Enable the m/c to think with out any human intervention, and take decision.

.

Artificial intelligence (AI) is an area of computer science that emphasizes the creation of **intelligent** machines that work and react like humans.

..

AI, is the use of programming to imitate human thought and action by analyzing data and surroundings, solving problems and learning to familiarize to a variety of tasks.



AI - Artificial Intelligence

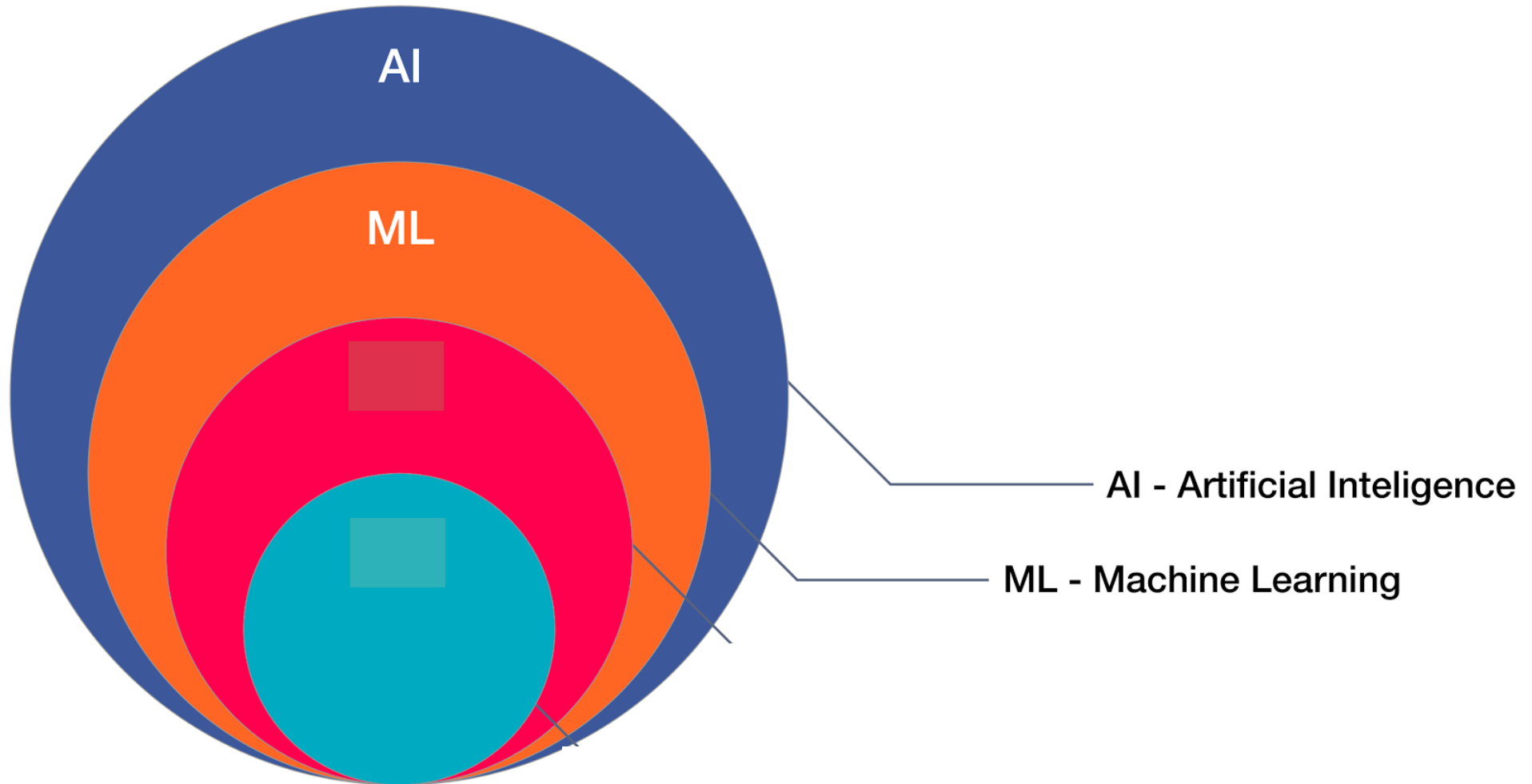
AI, ML, DL, NN and DS

Machine Learning

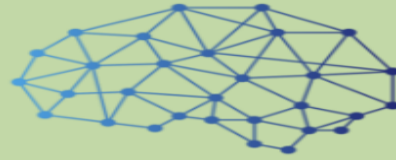
ML: M/C learning is the subset of AI, that allows machines to learn from data without being programmed explicitly. It provides statistical tools to explore and analyze the data.

ML is a method of data analysis that automates analytical model building.

It is a branch of AI based on the idea that systems can learn from data, identify patterns and make decisions with minimal human intervention.



Artificial Intelligence



Machine Learning



RL

Unsupervised learning



Supervised learning



Deep Learning



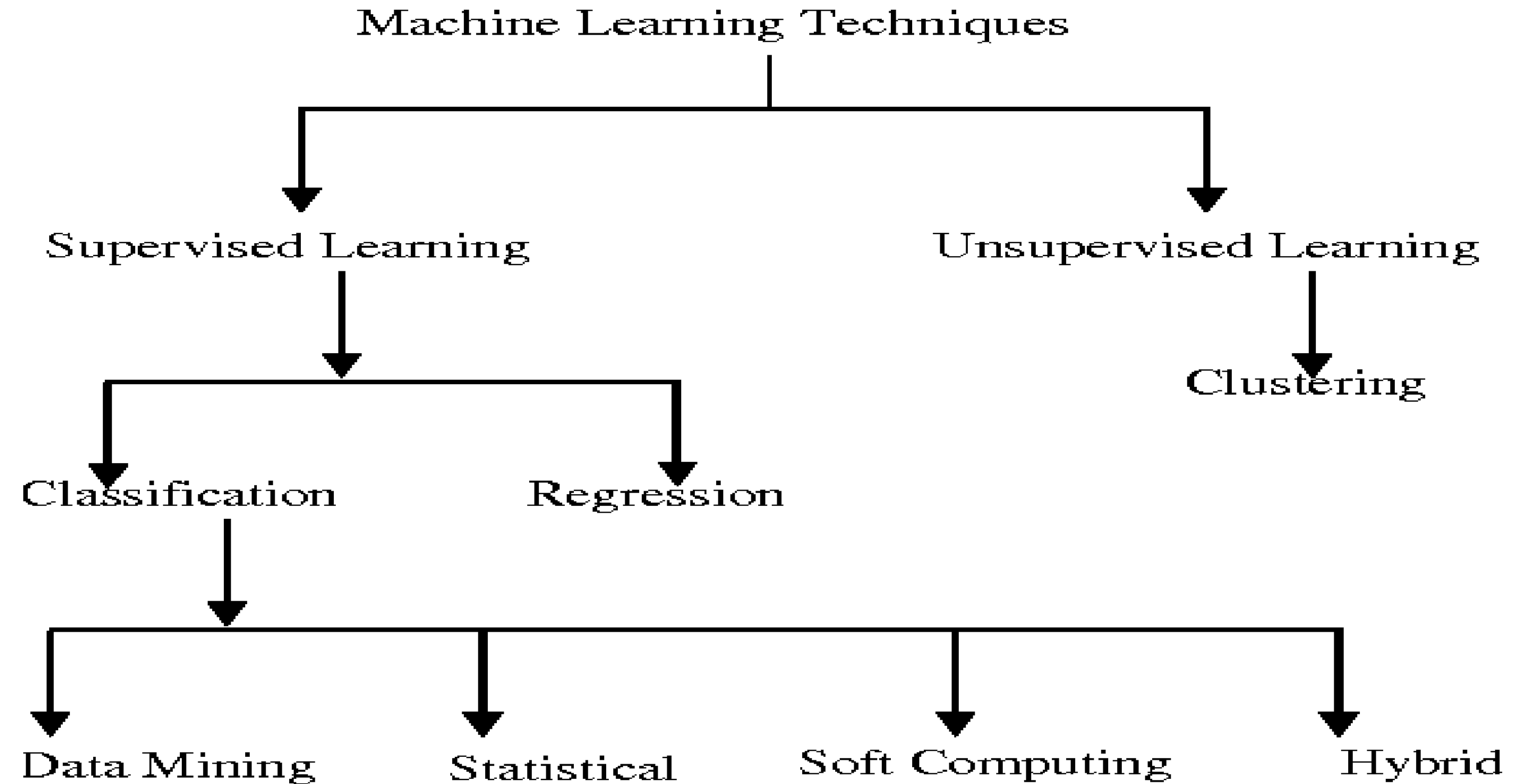


Figure 1. Classification of Machine Learning Techniques

AI, ML, DL, NN and DS

Neural networks

NN: make use of neurons that are used to transmit data in the form of input values and output values. They are used to transfer data by using networks of connections.

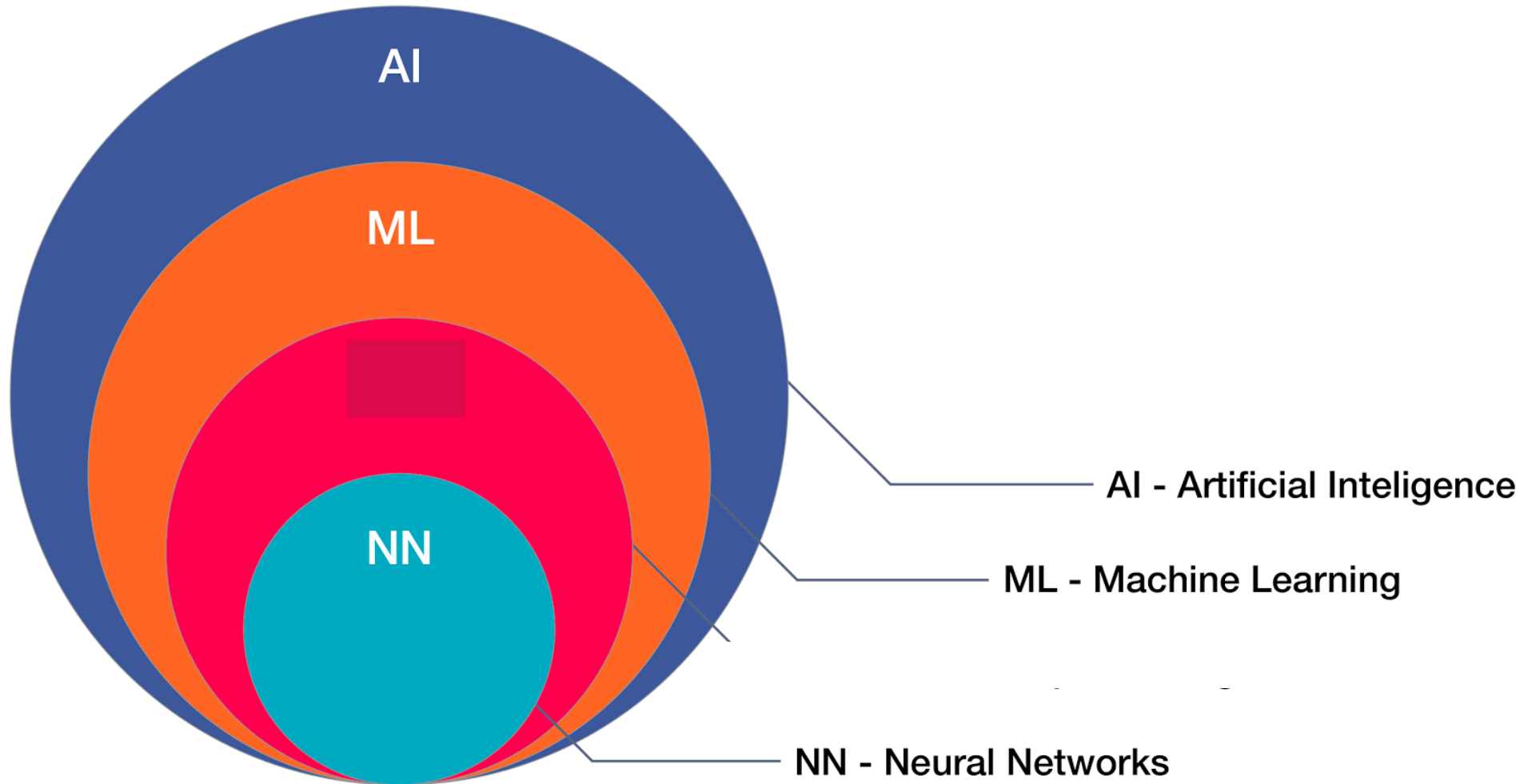
Artificial neural networks (ANNs) are machine learning techniques which simulate the functioning of human brain.

Artificial Neural networks consist of up to 3 layers namely

I/P layer

Hidden Layer

O/P layer



AI, ML, DL, NN and DS

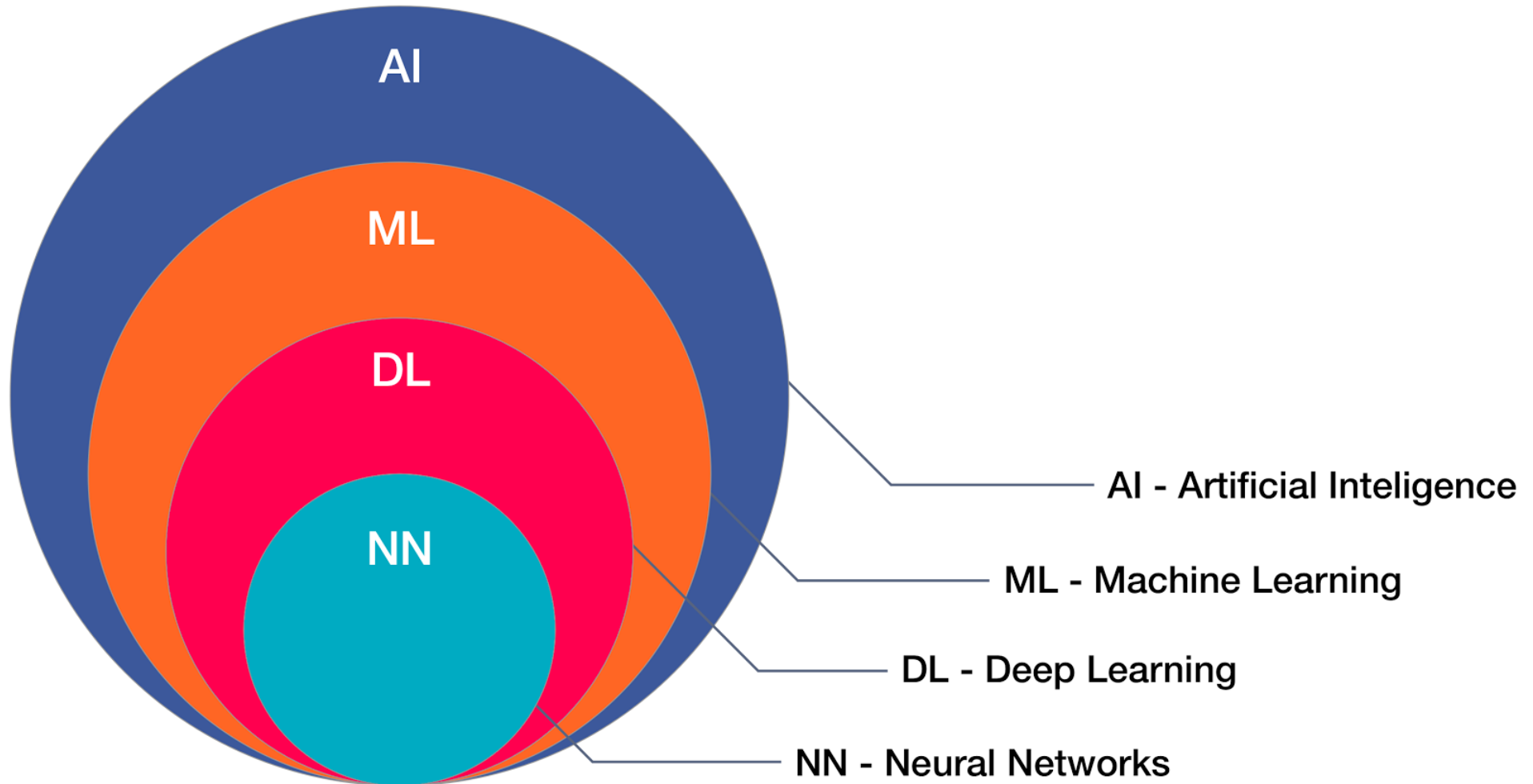
Deep learning

DL: composed of several hidden layers while neural networks consist of up to 3 layers.

DL is the subset of ML. This got created for the M/C to learn to think how actually human brain learn to think.

In DL, we create architecture called multi-neural network architecture. Idea of Deep neural network is to mimic human brain.

Eg: Neural Network and Deep Learning lies in the heart of products such as self driving cars, image recognition software etc.



AI, ML, DL, NN and DS

In deep Learning, we have 3 techniques:

ANNs : Problems that involved with the numbers are solved with the help of ANN.

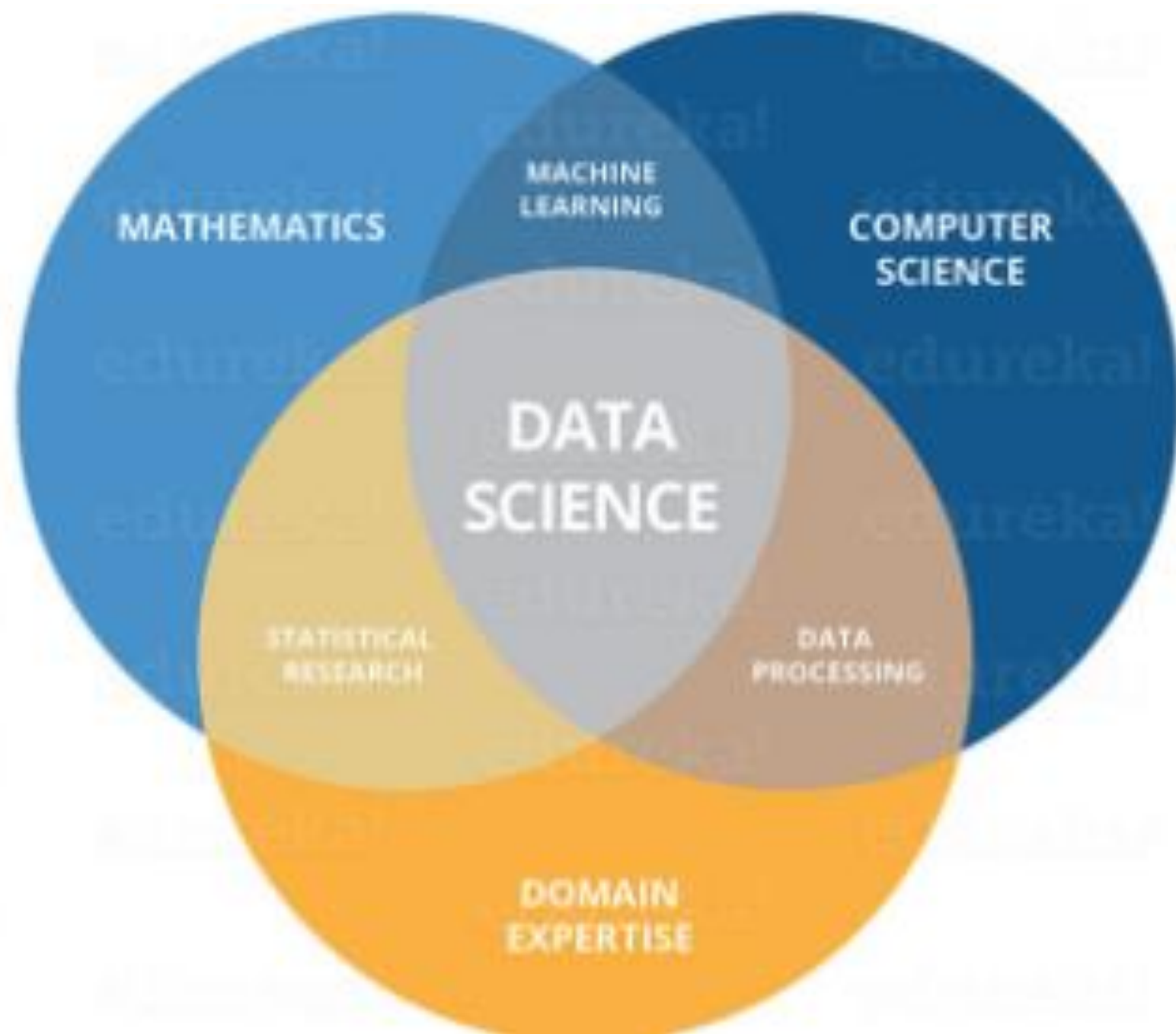
Convolution Neural Networks (CNN) : If the i/p is in images, then we use CNN. Advanced CNN known as Transfer learning.

Recurrent Neural Networks (RNN) : If data is time series kind of data, we use RNN.

AI, ML, DL, NN and DS

Data Science

DS: will work on all above AI, ML, DL using mathematical tools such as statistics, probability, Linear algebra, Differential calculus etc.



Introduction to Neural Networks

1.1 Introduction

Artificial Neural Networks (ANNs) are m/c learning techniques which simulate the functioning of human brain.

The human nervous system contains cells, referred as *neurons*.

The Biological Neuron

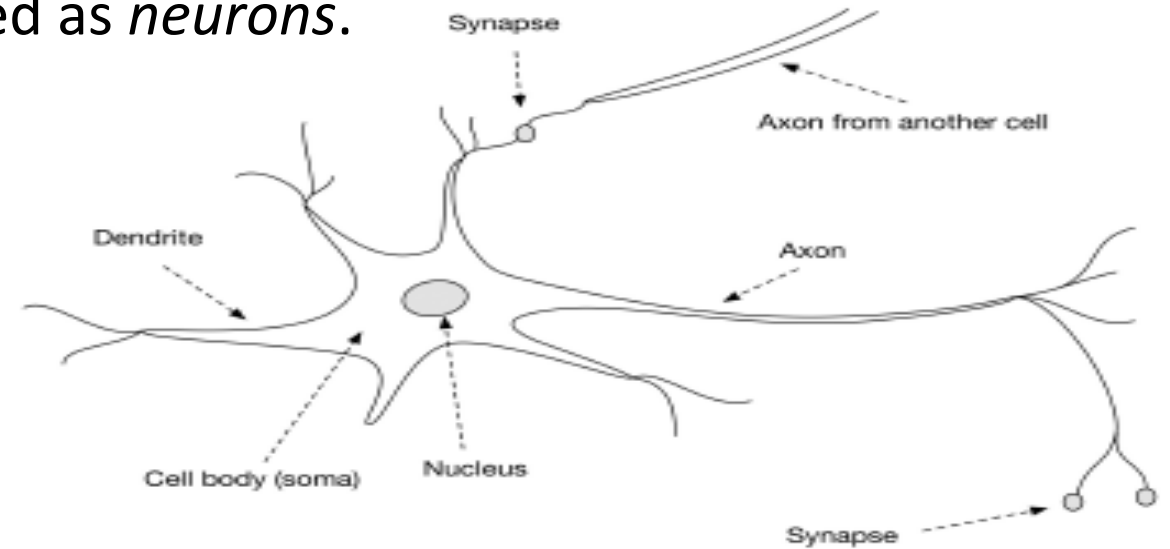


Figure 2-2. The biological neuron

The biological neuron (see Fig) is a nerve cell that provides the fundamental functional unit for the nervous systems of all animals.

Neurons exist to communicate with one another, and pass electro-chemical impulses across synapses, from one cell to the next.

The impulse must be strong enough to activate the next neuron by release of chemicals across a synaptic bifurcation.

The strength of the impulse must exceed a min threshold else chemicals will not be released.

The Biological Neuron (Contd..)

Fig. represents the major parts of the nerve cell:

Soma

Axons

Dendrites

Synapses

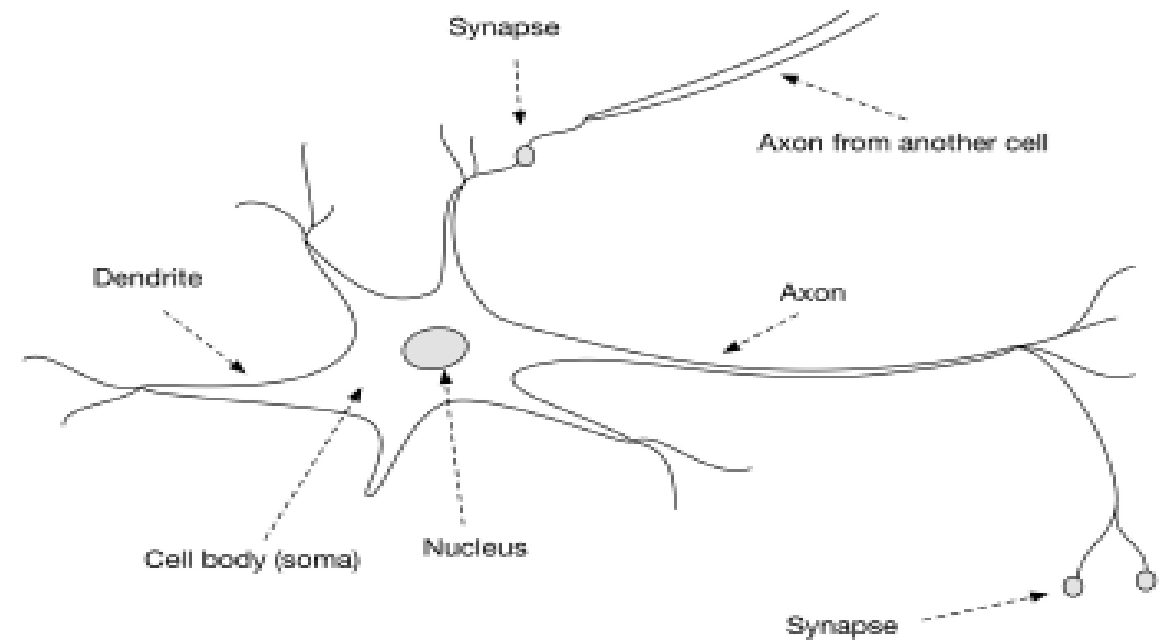


Figure 2-2. The biological neuron

The **neuron** is made up of a nerve cell consisting of a **soma** (cell body) that has many dendrites but only one axon.

Axons are nerve fibers with a special cellular extension that comes from the cell body. The single axon can branch hundreds of times, however.

Dendrites are thin structures that arise from the main cell body.

The Biological Neuron (Contd..)

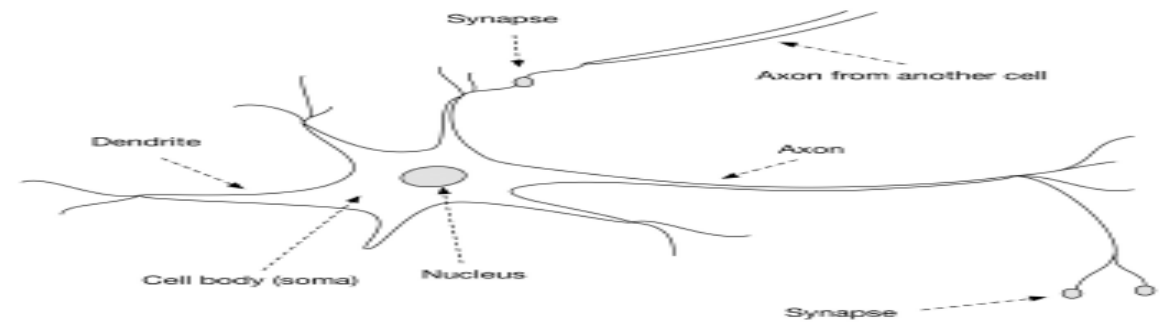


Figure 2-2. The biological neuron

Synapses

Synapses are the connecting junction b/n axon and dendrites.

The synapses send signals from the axon of a neuron to the dendrite of another neuron.

Dendrites

Dendrites have fibers branching out from the soma in a bushy network around the nerve cell.

Dendrites allow the cell to receive signals from connected neighboring neurons.

Each dendrite is able to perform multiplication by that dendrite's weight value.

Here multiplication means an increase or decrease in the ratio of signal chemicals introduced into the dendrite.

The Biological Neuron (Contd..)

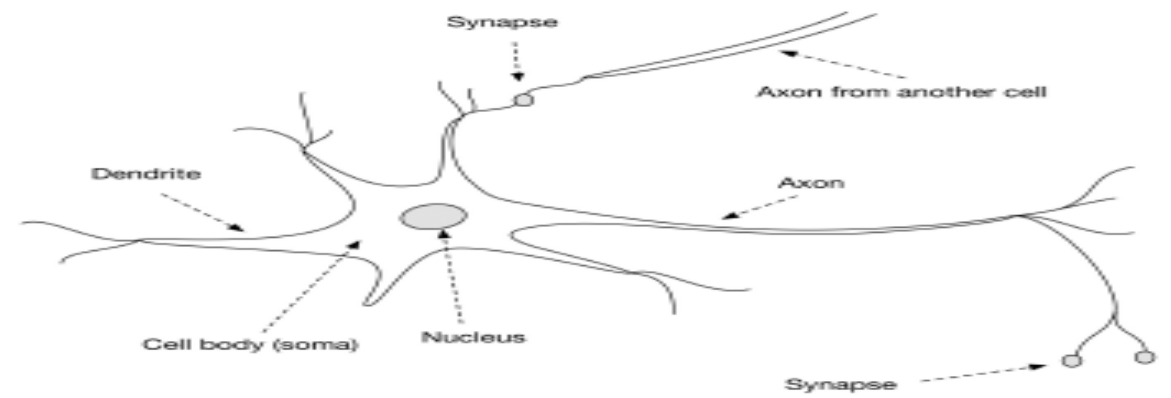


Figure 2-2. The biological neuron

Axons

Axons are the single, long fibers extending from the main soma.

They stretch out longer distances than dendrites and measure generally 1 cm in length (and 100 times the dia of the soma).

Eventually, the axon will branch hundreds of times and connect to other dendrites.

Neurons are able to send electrochemical signal that travels along the cell's axon and activates synaptic connections with other neurons.

Information flow across the biological neuron

Synapses that increase the potential and those that decrease the potential.

Neurons also have been shown to form new connections over time and even migrate.

These combined mechanics of connection change drive the learning process in the biological brain.

Total connections in the human brain

Biological neural networks (brains) are composed of roughly 86 billion neurons connected to many other neurons.

There are more than 500 trillion connections between neurons in the human brain.

Even the largest artificial neural networks today don't even come close to approaching this number.

From an information processing point of view, a biological neuron is an excitable unit that can process and transmit information via electro-chemical signals.

A neuron in the biological brain is considered a main component of the brain, spinal cord of the central nervous system.

Currently the **largest artificial neural networks**, built on supercomputers, have the size of a frog brain (about 16 million **neurons**).

1.1.1 Humans Versus Computers: Stretching the Limits of Artificial Intelligence

Humans and computers are inherently suited to different types of tasks.

For example, computing the cube root of a large number is very easy for a computer, but it is extremely difficult for humans.

On the other hand, a task such as recognizing the objects in an image is a simple matter for a human, but has traditionally been very difficult for an automated learning algorithm.

It is only in recent years that deep learning has shown an accuracy on some of these tasks that exceeds that of a human.

In fact, the recent results by deep learning algorithms that beat human performance in image recognition. It would not have been possible by most computer vision experts 10 years ago.

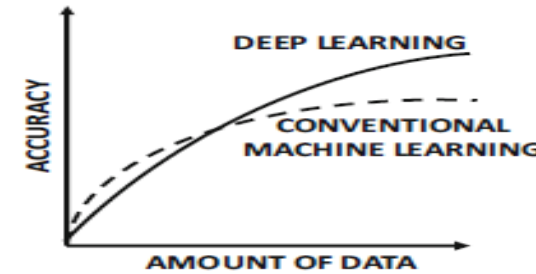
..

Furthermore, biological networks are connected in ways we do not fully understand.

Biological structure is understood at some level, significant innovations have been achieved by designing ANNs along those lines.

1.1.1 Humans Versus Computers: Stretching the Limits of Artificial Intelligence (Contd..)

The increased data availability and computational power of modern computers has outgrown the limits of traditional machine learning algorithms.



In above Fig., the performance of traditional machine learning remains better for smaller data sets.

The “big data” era has been enabled by the advances in data collection technology.

Furthermore, the development of powerful GPUs has enabled efficient processing on such large data sets.

These advances largely explain the recent success of deep learning algorithms.

The deep learning algorithms have been enabled by increased speed of computation, and because of reduced run-times enable efficient testing.

If it requires a month to test an algorithm, at most twelve variations can be tested in an year.

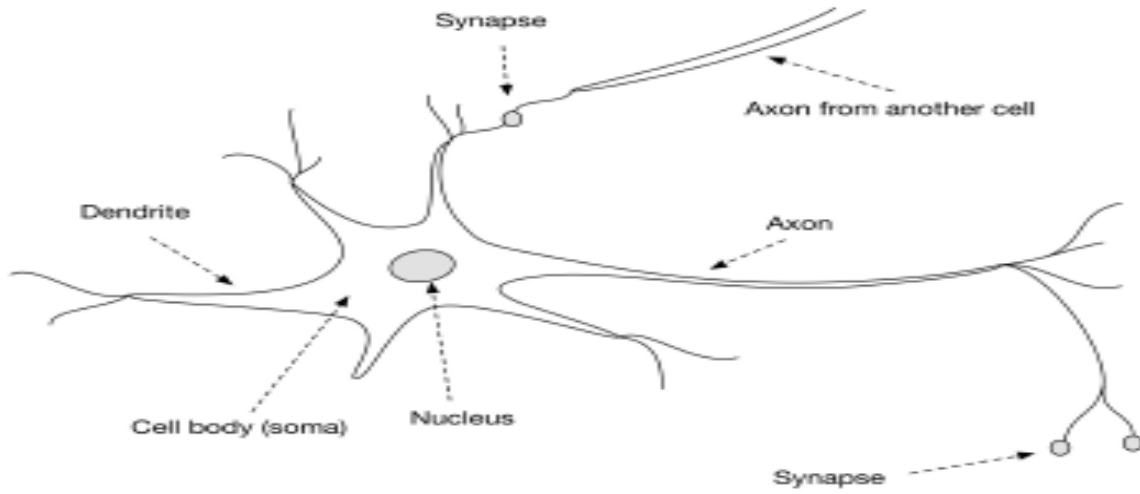


Figure 2-2. The biological neuron

Fig. Biological neuron

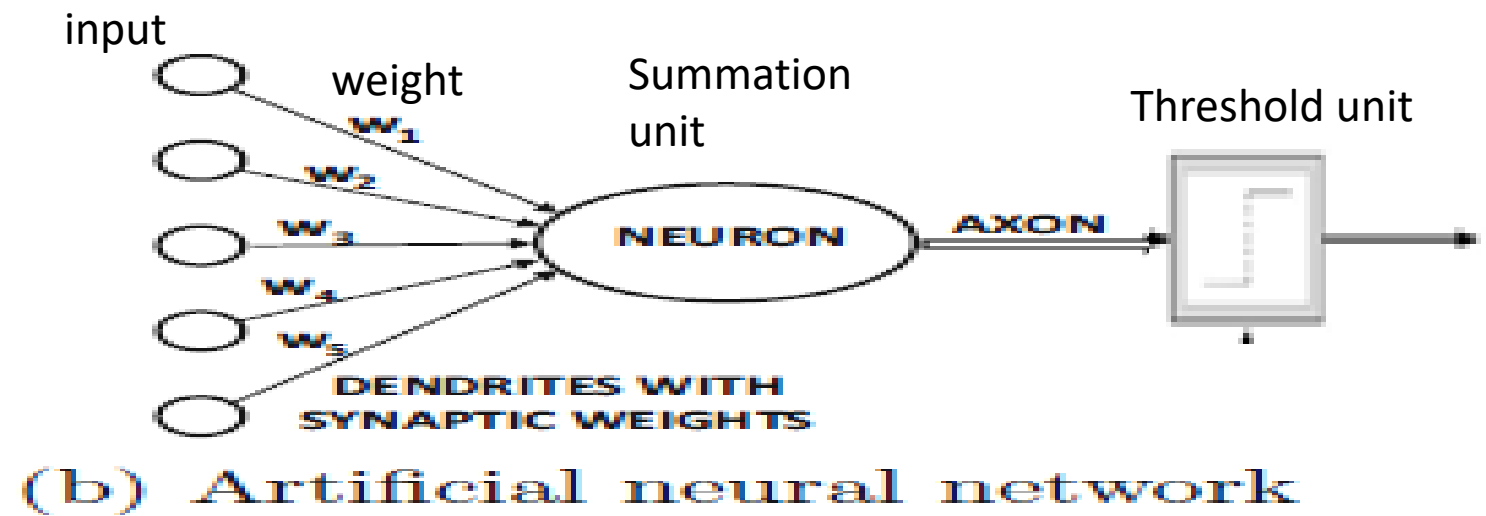
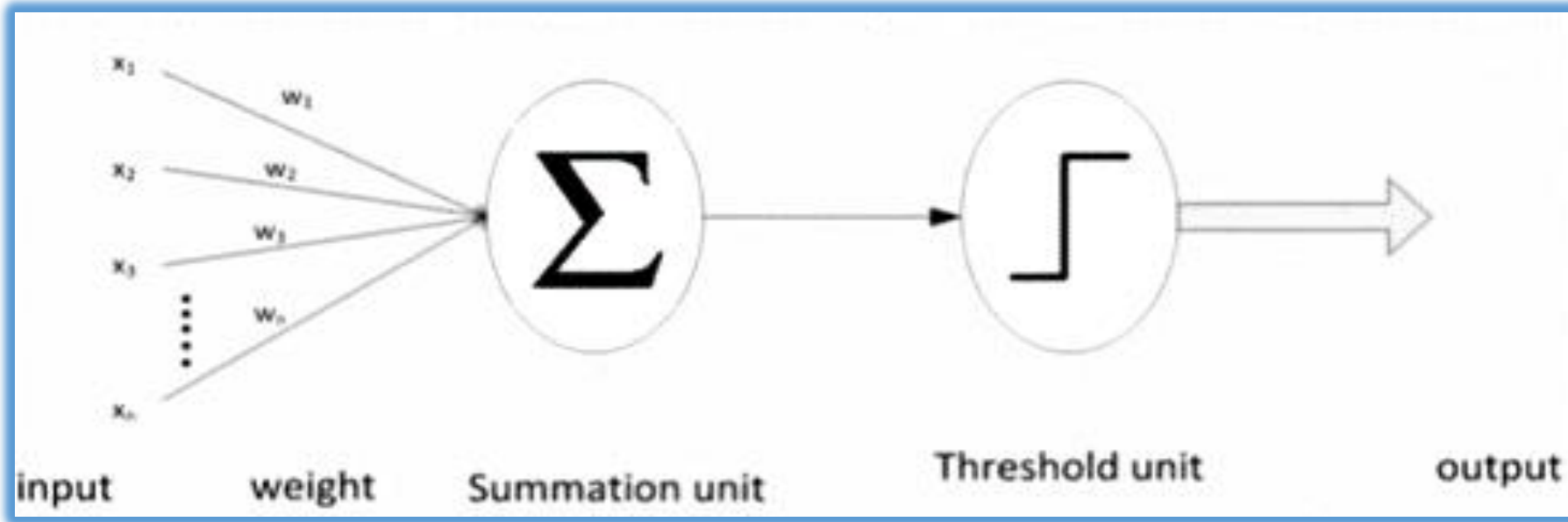


Fig Artificial neural network



Mimic the biological neuron to artificial neuron called Perceptron.

$x_1, x_2, \dots, x_n$ are the i/p's. Summation unit. I/p's coming with weights. Weight is equivalent to electro-chemical impulse in biological neuron.

All the i/p's multiplied with their resp weights are summed up in summation unit as $Z = W_1X_1 + W_2X_2 + \dots + W_nX_n$.

$$= W_1X_1 + W_2X_2 + \dots + W_nX_n = \sum_{i=1}^n W_iX_i$$

At threshold unit the summation result is checked for whether it is $>$ than a threshold or less. If $>$ then that signal will pass to next neuron and if less it will not pass.

A simple for loop for summation + if then else condition for threshold.

To get the final o/p y , the sum is passed to the threshold unit which uses a filter called activation function i. e. $y = f(Z)$

Use of additional input, which is called **bias**, of neuron improves properties of the neuron. It allows moving the threshold of activation function.

The **bias value** allows the activation function to be shifted to the left or right, to better fit the data. It is always 1.

A **weight** represent the strength of the connection between units. If the **weight** from node 1 to node 2 has greater magnitude, it means that **neuron 1** has greater influence over **neuron 2**.

Weights in NN **can**, and **will**, become positive or **negative** depending on the training data.

The **training** function is the overall algorithm that is used to train the **neural network** to recognize a certain input and map it to an output.

The **training** process involves finding a set of weights in the **network** that proves to be good, or good enough, at solving the specific problem.

A **learning** function deals with individual weights and thresholds and decides how those would be manipulated.

Activation function decides, whether a neuron should be **activated** or not by calculating weighted sum and further adding bias with it.

Sigmoid Activation function: It is a activation function of form $f(z) = 1 / (1 + e^{-z})$.

Its Range is between 0 and 1.

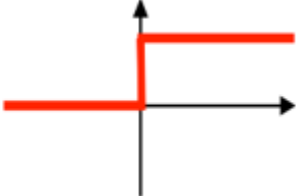
It is easy to understand and apply.

Activation Functions:

Unit step
(Heaviside)

$$\phi(z) = \begin{cases} 0, & z < 0, \\ 0.5, & z = 0, \\ 1, & z > 0, \end{cases}$$

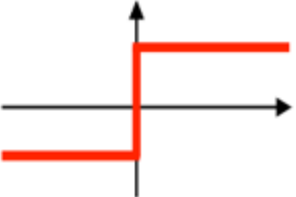
Perceptron
variant



Sign (Signum)

$$\phi(z) = \begin{cases} -1, & z < 0, \\ 0, & z = 0, \\ 1, & z > 0, \end{cases}$$

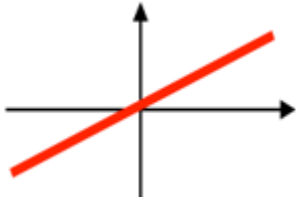
Perceptron
variant



Linear
or
Identity

$$\phi(z) = z$$

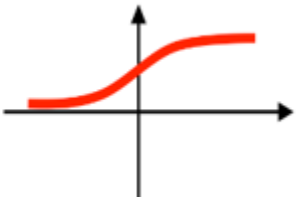
Adaline, linear
regression



Logistic (sigmoid)

$$\phi(z) = \frac{1}{1 + e^{-z}}$$

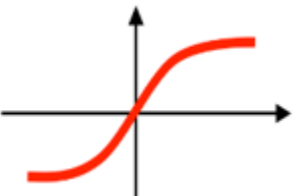
Logistic
regression,
Multi-layer NN



Hyperbolic tangent

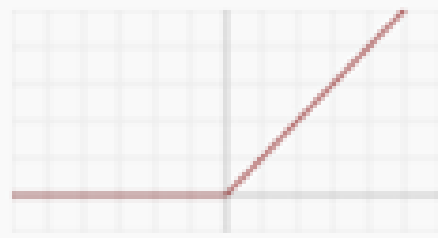
$$\phi(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

Multi-layer NN



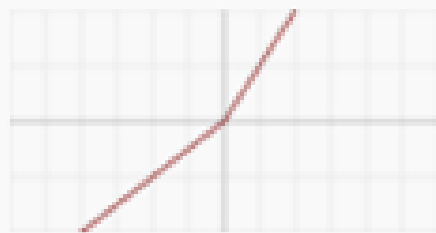
Activation Functions:

Rectified
Linear Unit
(ReLU)



$$f(\mathbf{z}) = \begin{cases} 0 & \text{for } \mathbf{z} < 0 \\ \mathbf{z} & \text{for } \mathbf{z} \geq 0 \end{cases}$$

Parameteric
Rectified
Linear Unit
(PReLU) [\[2\]](#)



$$f(\mathbf{z}) = \begin{cases} \alpha \mathbf{z} & \text{for } \mathbf{z} < 0 \\ \mathbf{z} & \text{for } \mathbf{z} \geq 0 \end{cases}$$

Working of Neural Network: Forward propagation

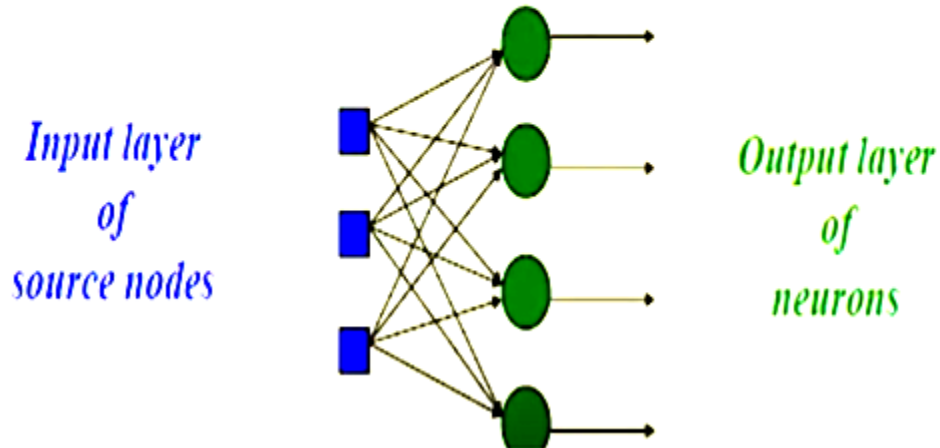
To train Neural Network with Back Propagation

McCulloch-Pitts Model

Network Architectures

- Two different classes of network architectures
 - Single-layer feed-forward
 - Multi-layer feed-forward
- The architecture of a neural network is linked with the learning algorithm used to train

Single Layer Feed-forward



There are two different classes of network architecture:
Single layer feed forward architecture
Multi-layer feed forward architecture
The architecture that we use depends upon the learning algorithm that you are going to use train the given network.

Single layer feed forward network:

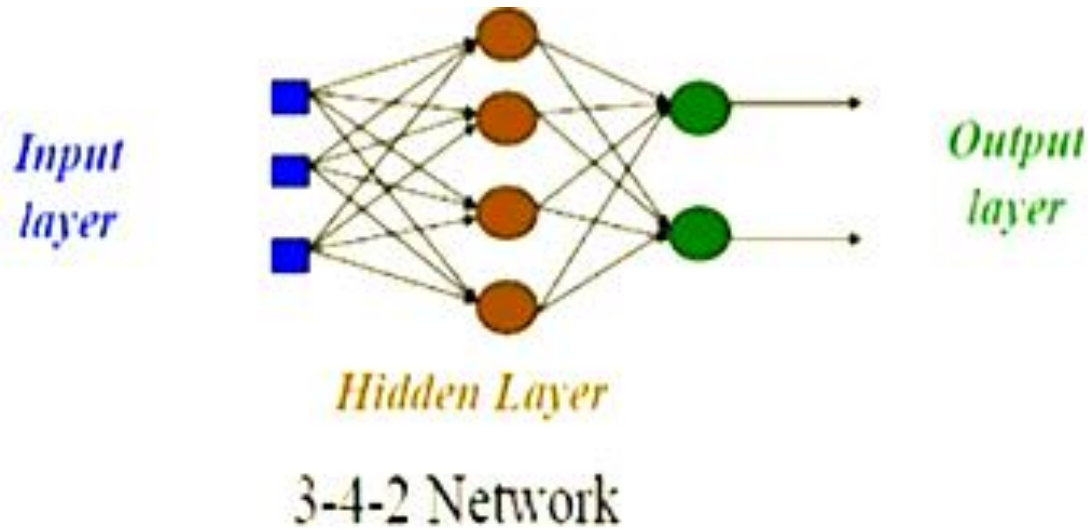
In this network there are the input layer and there is the one output layer.

So in input layer neurons or the nodes which are present in the input layer are just to receiving the input and output layer neurons will be processing the input by taking the input from the input layer.

So as such input layer it is the dummy layer which is not processing or which is not doing anything for the input. It just take the input and apply it to the output layer.

The problem with the single layer feed forward network is that it cannot be used for solving the problems of the nonlinearity.

Multi-layer FFNN:



Multilayer feed forward neural network is the most general neural network architecture where there are the hidden layer between the input and output.

We are adding the additional layer which is called as the hidden layer.

Actually the hidden layer is not receiving the any input directly or it is not giving the output directly.

It is receiving the input for processing through the input layer then it process that input and then it will produce output through the output layer.

Multilayer feedforward network is able to handle the nonlinear separability linear task.

Linear Separability

Graph for AND

Graph for OR

Graph for Ex-OR

To understand the linear separability concept we will consider the basic functions: AND, OR and XOR. Consider the graphical representation of them.

So with the AND case there exist a single straight line or the linear line which separate this output into two classes. i.e. the class 1 and class 0.

So with OR, again a single straight line exist which separate out this output into the two classes.

So AND and OR, NAND, NOR are called to be linear function because there exist a single straight line which separate this output into two classes. i.e. the class 1 and class 0.

In case of XOR gate, you cannot draw single line to separate out these output into the two classes. That's why XOR is called to be linearly non separable function. Still you can draw the two lines for this to separate out.

McCulloch Pitts Neuron Model

- How to design Logic gates?

 - OR, AND, NOT, XOR

 - NAND, NOR, XNOR

How to use NN to implement this simple logic of OR, AND gates:

McCulloch-Pitts Model

We can implement the OR function by using the McCulloch-Pitt model.
Here it is the basic model of neuron, which doesn't contain having any bias.

TT of OR

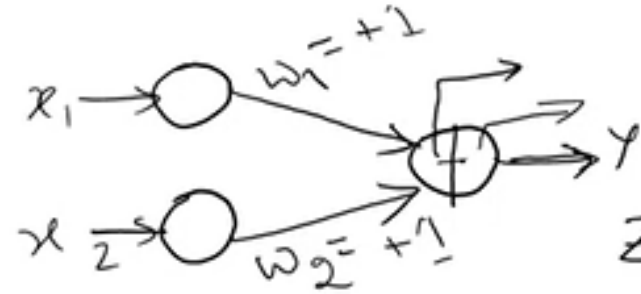
NN model

Calculate the output:

- i) Compute Z
- ii) Compute Y after applying activation function

Draw a conclusion

OR Gate



$$z = w_1 x_1 + w_2 x_2$$

$$y = \text{Act}(z)$$

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	1

if x_1, x_2 0 0 then

$$z = 1 \times 0 + 1 \times 0 = 0$$

$$0 \ 1 \quad z = 1 \times 0 + 1 \times 1 = 1$$

$$1 \ 0 \quad = 1$$

$$1 \ 1 \quad = 2$$

$$\theta = +1$$

$$\begin{matrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{matrix} \rightarrow \begin{matrix} f(z) = 0 \\ f(z) = 1 \\ f(z) = 1 \\ f(z) = 1 \end{matrix}$$

AND Gate

$W1 =$

$W2 =$

$\theta =$

NOT Gate

$W1 =$

$\theta =$

XOR Gate:

XOR gate: $x_1 \oplus x_2 = x_1 \bar{x}_2 + \bar{x}_1 x_2$

$x_1 \bar{x}_2 = x_1 \text{ AND } \bar{x}_2$

x_1	x_2	$\bar{x}_2$	$x_1 \bar{x}_2$
0	0	1	0
0	1	0	0
1	0	1	1
1	1	0	0

$\bar{x}_1 x_2 = \bar{x}_1 \text{ AND } x_2$

x_1	x_2	$\bar{x}_1$	$\bar{x}_1 x_2$
0	0	1	0
0	1	1	1
1	0	0	0
1	1	0	0

$$W_{11}^1 = +1$$

$$W_{21}^1 = -1$$

$$\theta = +1$$

$$W_{12}^1 = -1$$

$$W_{22}^1 = +1$$

$$\theta = +1$$

$$x_1 \bar{x}_2 = x_1 \text{ AND } \bar{x}_2$$

$$W_{11}^1 = +1$$

$$W_{21}^1 = -1$$

$$\theta = +1$$

$$\bar{x}_1 x_2 = \bar{x}_1 \text{ AND } x_2$$

$$W_{12}^1 = -1$$

$$W_{22}^1 = +1$$

$$\theta = +1$$

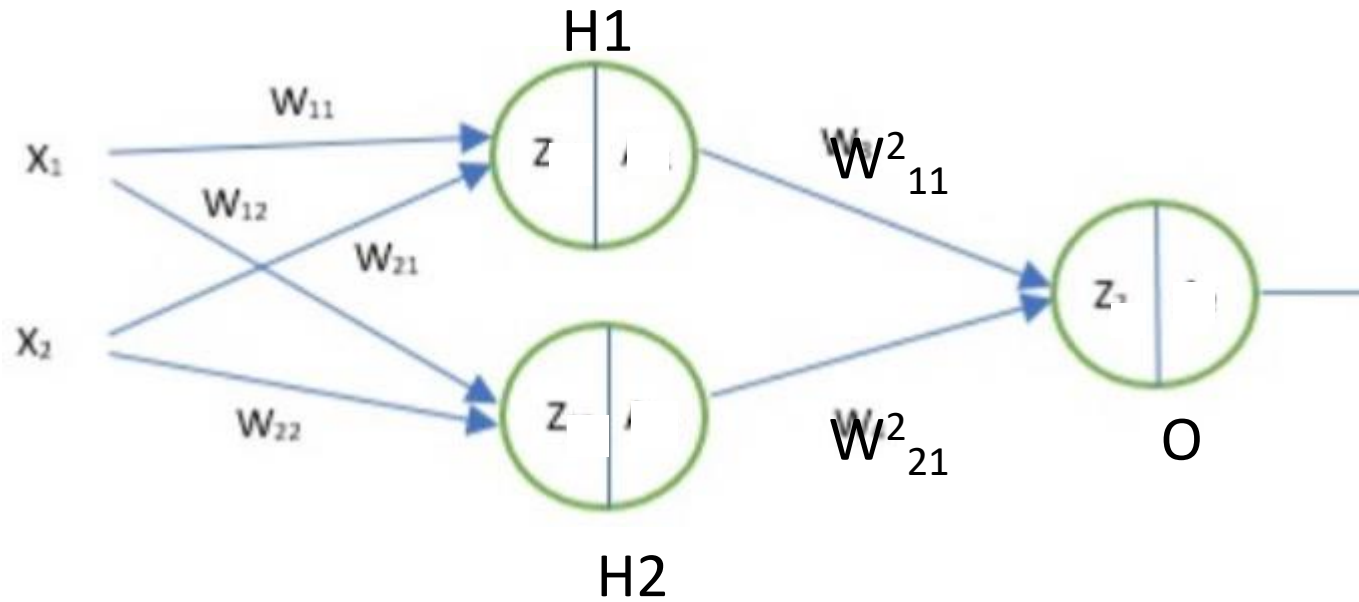
$$\text{XOR gate: } x_1 \oplus x_2 = x_1 \bar{x}_2 + \bar{x}_1 x_2$$

$$W_{11}^2 = +1$$

$$W_{21}^2 = +1$$

$$\theta = +1$$

Design X-OR gate . Use the sigmoid activation function.



$$\phi(z) = \frac{1}{1 + e^{-z}}$$

In the above diagram,

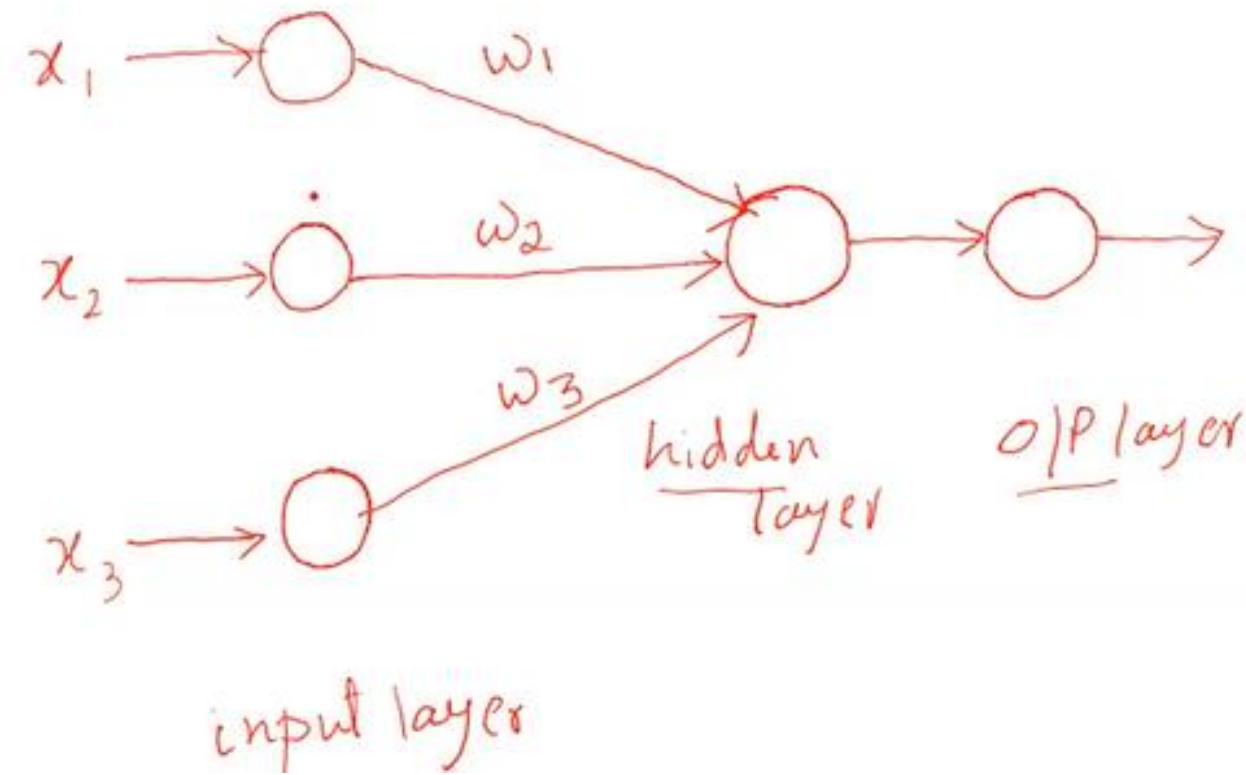
$W'_{11} = W'_{21} = W'_{12} = W'_{22} = 1$ and

$W^2_{11} = 1$ $W^2_{21} = -1$

At Node $H1$, $\theta = 0.5$

At Node $H2$, $\theta = 1.5$

At Node O , $\theta = 0.2$



Forward path:

Let $x_1 = 1$ $x_2 = 1$ $x_3 = 1$

And $W_{11}^1 = 0$ $W_{21}^1 = 1$ $W_{31}^1 = 1$ $W_{11}^2 = 1$

Assume sigmoid is the activation function

Step 1:

$Z_1 = 2$ (i.e. $W_{11}^1 x_1 + W_{21}^1 x_2 + W_{31}^1 x_3 = 0 \times 1 + 1 \times 1 + 1 \times 1 = 2$)

Step 2:

$$O_{11} = f(z) = 1 / (1 + e^{-z}) = 0.8808$$

Similarly

Step 1,

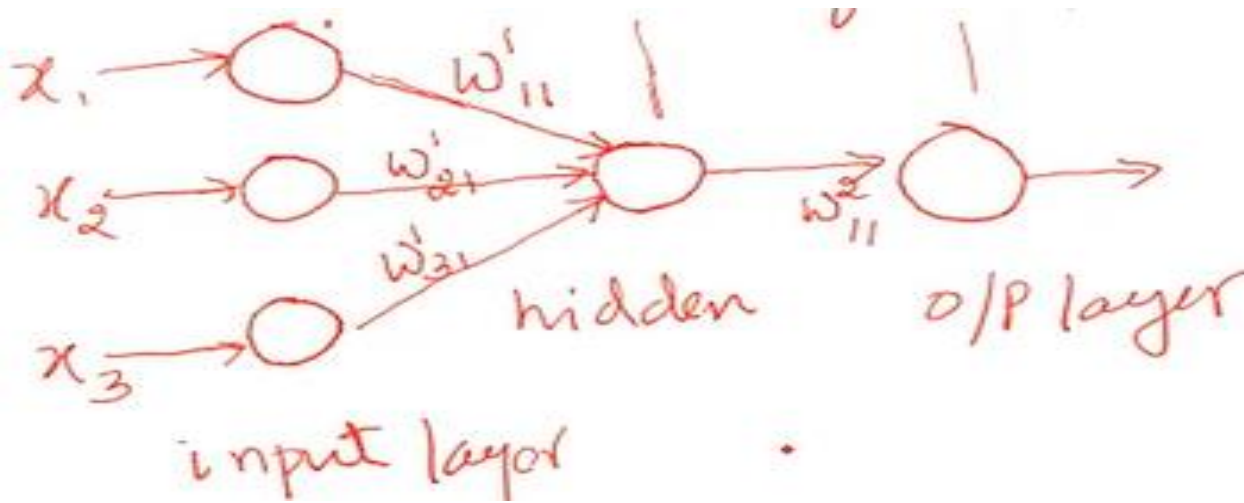
$$Z_2 = W_{11}^2 O_{11} = 0.8808$$

Step 2,

$$O_{21} = f(z) = 1 / (1 + e^{-z}) = 0.707$$

Having known with the values:

Chain Rule of differentiation with Back propagation



$$w_{11}^2_{\text{new}} = w_{11}^2_{\text{old}} - \eta \frac{\partial L}{\partial w_{11}^2}$$

$$\frac{\partial L}{\partial w_{11}^2} = \frac{\partial L}{\partial o_{21}} \times \frac{\partial o_{21}}{\partial w_{11}^2}$$

$$\text{Now } \frac{\partial L}{\partial o_{21}} = \frac{\partial (Y - o_{21})^2}{\partial o_{21}} = 2(Y - o_{21})(-1) = -2(Y - o_{21})$$

$$\frac{\partial o_{21}}{\partial w_{11}^2} = \frac{\partial o_{21}}{\partial z_2} \times \frac{\partial z_2}{\partial w_{11}^2}$$

$$z_2 = w_{11}^2 o_{11}$$

$$o_{21} = \text{Act}(z_2)$$

$$\frac{\partial o_{21}}{\partial z_2} = \frac{\partial [\text{Act}(z_2)]}{\partial z_2} = \text{Act}(z_2) \times [1 - \text{Act}(z_2)]$$

$$\frac{\partial z_2}{\partial w_{11}^2} = o_{11}$$

$$\frac{\partial o_{21}}{\partial w_{11}^2} = \text{Act}(z_2) \times (1 - \text{Act}(z_2)) \times o_{11}$$

$$\frac{\partial L}{\partial w_{11}^2} = -2(Y - o_{21}) \times \text{Act}(z_2) \times (1 - \text{Act}(z_2)) \times o_{11}$$

in eqn (1)

$$w_{11}^2_{\text{new}}$$

Having known with the values:

$$\text{Let } x_1 = 1 \quad x_2 = 1 \quad x_3 = 1$$

$$\text{And } w_{11}^1 = 0 \quad w_{21}^1 = 1 \quad w_{31}^1 = 1 \quad w_{11}^2 = 1$$

$$z_1 = 2 \quad o_{11} = 0.8808$$

$$z_2 = 0.8808 \quad o_{21} = 0.707$$

Back propagation

$$w_{11}^{\text{new}} = w_{11}^{\text{old}} - \eta \left(\frac{\partial L}{\partial w_{11}} \right)$$

$$\frac{\partial L}{\partial w_{11}} = \frac{\partial L}{\partial o_{21}} \times \frac{\partial o_{21}}{\partial z_2} \times \frac{\partial z_2}{\partial o_{11}} \times \frac{\partial o_{11}}{\partial z_1} \times \frac{\partial z_1}{\partial w_{11}} \rightarrow \text{chain rule}$$

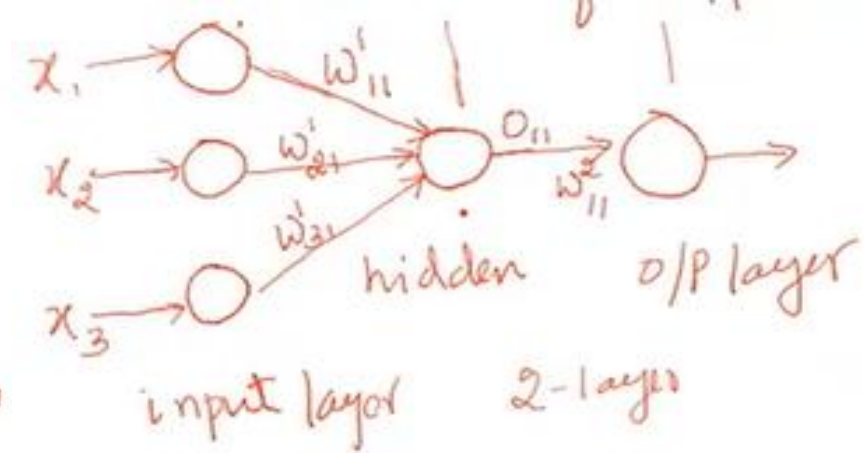
$$\hookrightarrow \frac{\partial [w_{11}'x_1 + w_{21}'x_2 + w_{31}'x_3]}{\partial w_{11}'} = x_1 + 0 + 0 = x_1$$

$$\hookrightarrow \frac{\partial [\text{Act}(z_1)]}{\partial z_1} = \text{Act}(z_1) \times (1 - \text{Act}(z_1))$$

$$\hookrightarrow \frac{\partial [w_{11}^2 o_{11}]}{\partial o_{11}} = w_{11}^2$$

$$\hookrightarrow \frac{\partial (\text{Act}(z_2))}{\partial z_2} = \text{Act}(z_2) \times (1 - \text{Act}(z_2))$$

$$\hookrightarrow \frac{\partial (y - o_{21})^2}{\partial o_{21}} = -2(y - o_{21})$$



Having known with the values:

Let $x_1 = 1$ $x_2 = 1$ $x_3 = 1$

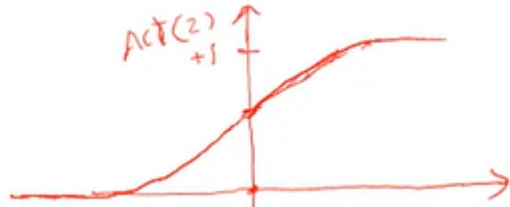
And $w_{11}^1 = 0$ $w_{21}^1 = 1$ $w_{31}^1 = 1$ $w_{11}^2 = 1$

$z_1 = 2$ $o_{11} = 0.8808$

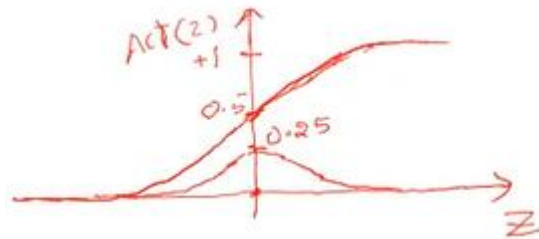
$z_2 = 0.8808$ $o_{21} = 0.707$

Derivative of Activation function

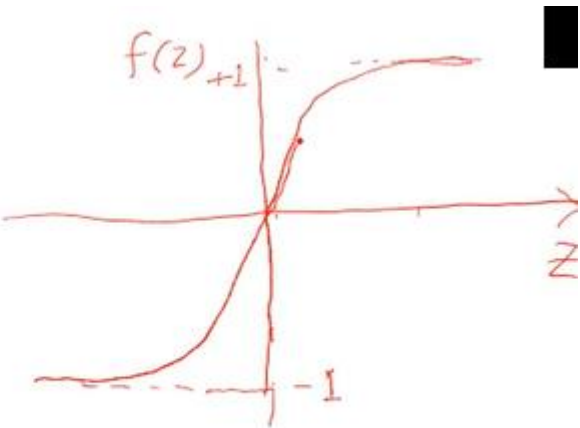
1) Sigmoid = $\frac{1}{1+e^{-z}}$



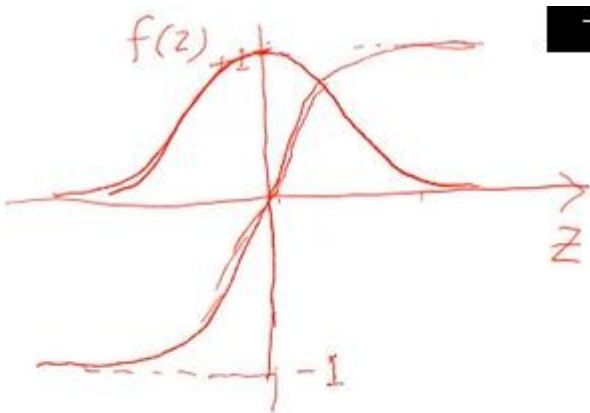
Derivative of Activation function Sigmoid: Max value 0.25



2) Tanh = $\frac{e^z - e^{-z}}{e^z + e^{-z}}$



Derivative of Activation function Tan hyperbolic: Max value 1

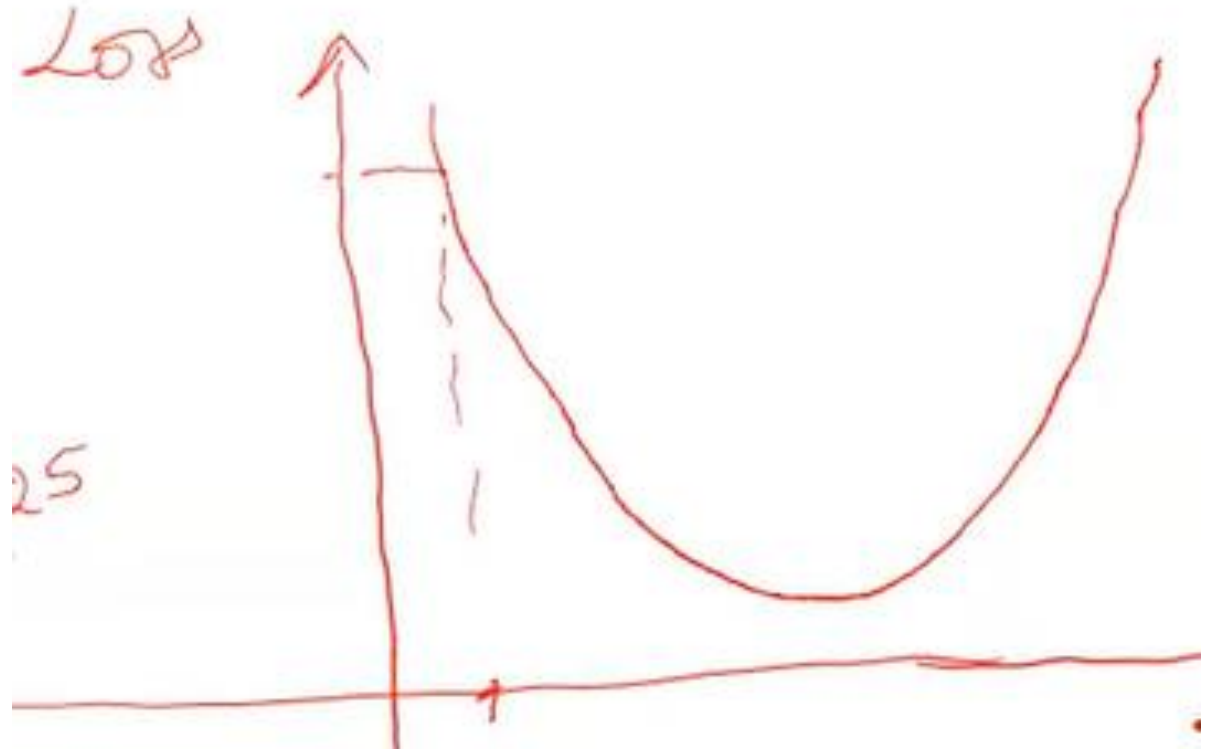


Vanishing gradient problem in a ANN

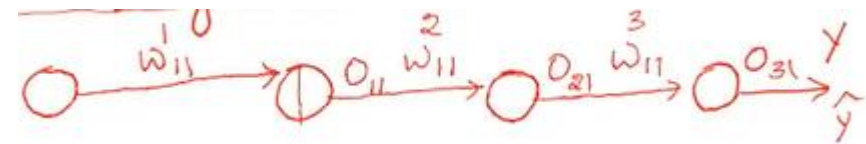


$$w_{11}^1_{\text{new}} = w_{11}^1_{\text{old}} - \eta \frac{\partial L}{\partial w_{11}^1}$$

$$\frac{\partial L}{\partial w_{11}^1} = \frac{\partial L}{\partial o_{21}} \times \frac{\partial o_{21}}{\partial o_{11}} \times \frac{\partial o_{11}}{\partial w_{11}^1}$$



Exploding gradient problem in a ANN



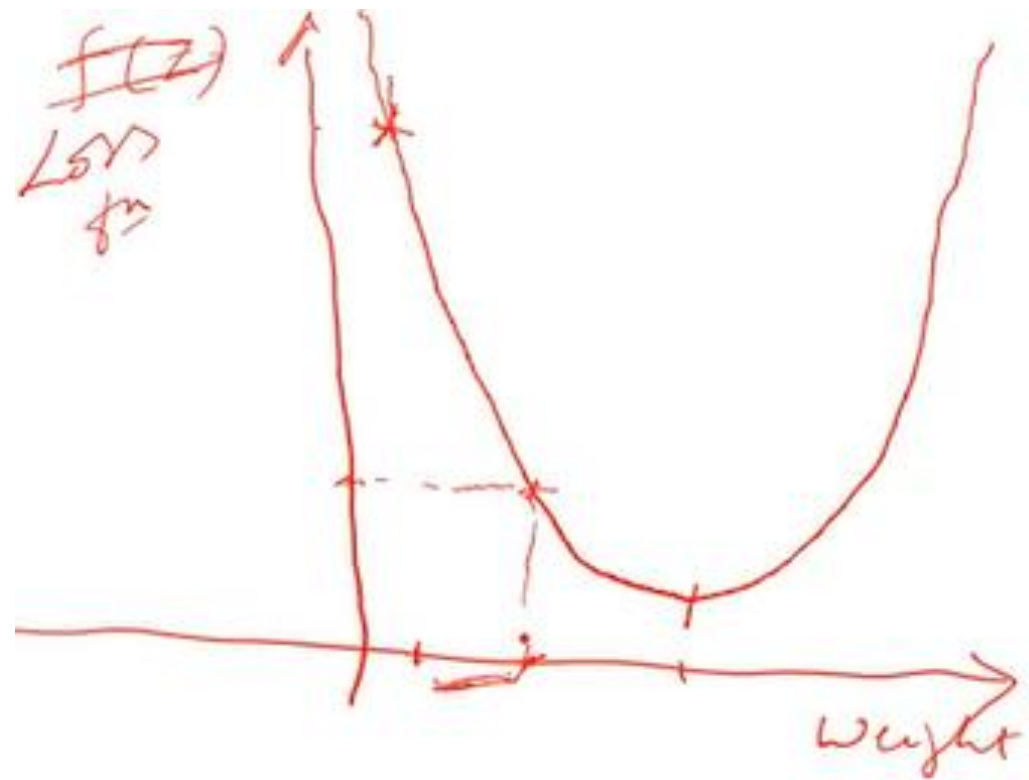
$$w'_{11} = w_{11} - \eta \frac{\partial L}{\partial w'_{11}}$$

$$\frac{\partial L}{\partial w'_{11}} = \frac{\partial L}{\partial o_{31}} \times \frac{\partial o_{31}}{\partial o_{21}} \times \frac{\partial o_{21}}{\partial o_{11}} \times \frac{\partial o_{11}}{\partial w'_{11}}$$

not just because of act. fn. but weights

$$\frac{\partial o_{21}}{\partial o_{11}} = \frac{\partial \text{Act}(z_2)}{\partial z_2} \times \frac{\partial z_2}{\partial o_{11}}$$

$$(\overline{0.0025}) \times \frac{\partial [w_{11}^2 \times o_{11}]}{\partial o_{11}}$$



$$\frac{\partial o_{31}}{\partial o_{21}} \text{ \& \& } \frac{\partial o_{11}}{\partial w'_{11}}$$

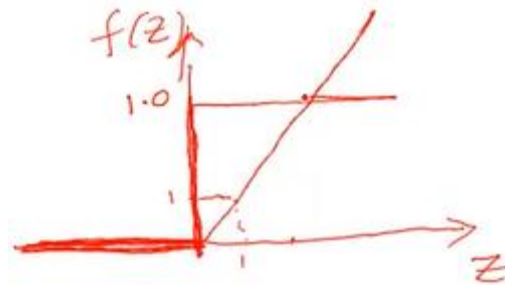
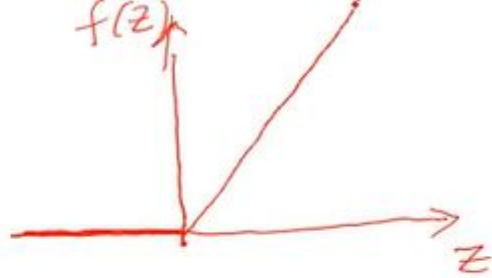
$$w_{11}^{\text{new}} \stackrel{\text{vary}}{=} w_{11}^{\text{old}}$$

1.0 -50

↓

3) ReLU = $f(z) = 0$ for $z < 0$
 $= z$ for $z \geq 0$

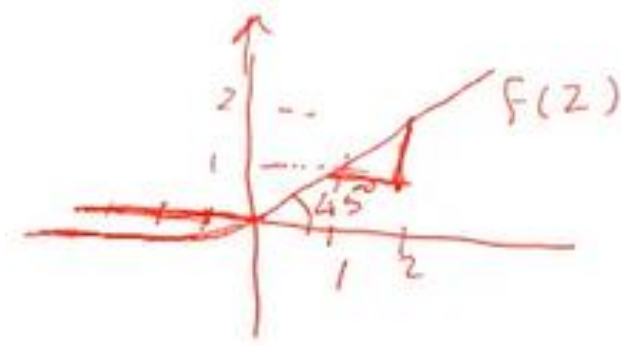
$= \max(z, 0)$



$\frac{\partial \text{Relu}(z)}{\partial z} =$ ~~0 or 1~~
 0 for $z < 0$
 1 for $z \geq 0$

$$f(Z) = \alpha Z \quad \text{for } Z < 0$$

$$= Z \quad \text{for } Z \geq 0$$



$$\tan 45^\circ = 1$$

$$\frac{\partial f(Z)}{\partial Z} = 1$$

$$W_{11}^{new} = W_{11}^{old} - \eta \frac{\partial L}{\partial W_{11}}$$

$$\frac{\partial L}{\partial W_{11}'} = \frac{\partial L}{\partial O_{21}} \times \frac{\partial O_{21}}{\partial O_{11}} \times \frac{\partial O_{11}}{\partial W_{11}'}$$

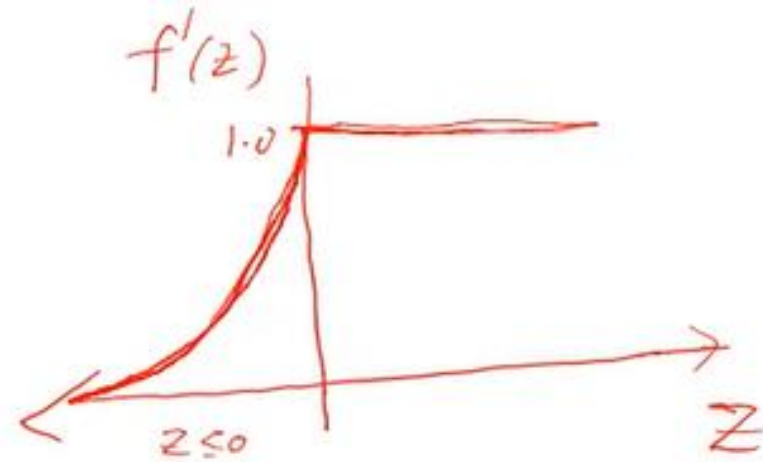
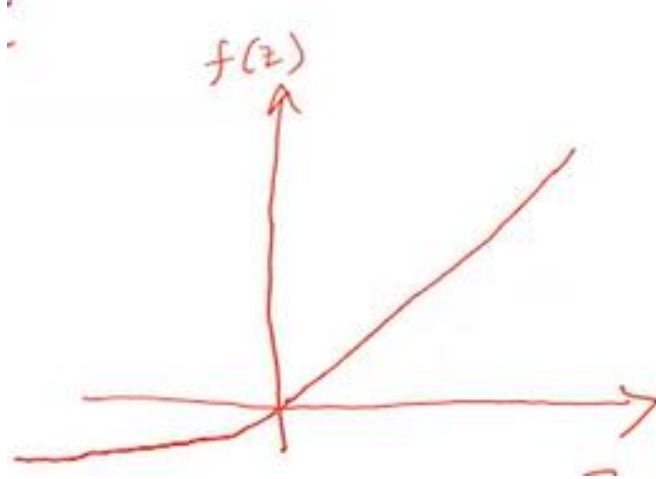
$\alpha_{0.01} \quad \times 0.01$

$$\frac{\partial f(Z)}{\partial Z} = \alpha \quad Z < 0$$

Leaky ReLU

ELU (Exponential Linear Unit)

$$f(z) = \begin{cases} \alpha(e^z - 1) & \text{if } z \leq 0 \\ z & \text{if } z > 0 \end{cases}$$



PReLU (Parametric ReLU)

✓ $f(z) = \begin{cases} \alpha z & \text{if } z < 0 \\ z & \text{if } z \geq 0 \end{cases}$

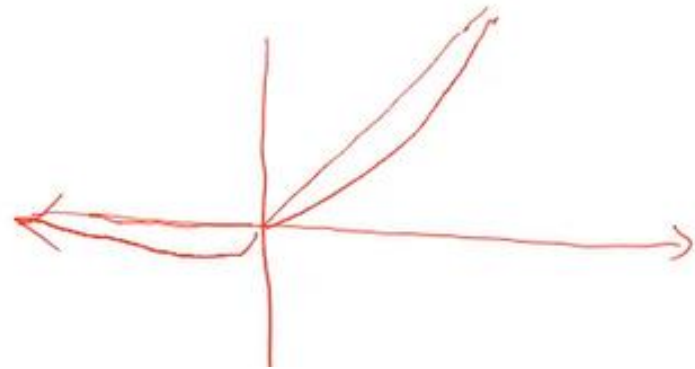
• $\alpha = 0.01 \rightarrow$ static $\rightarrow$ Leaky ReLU

$\alpha = 0 \rightarrow$ ReLU

α is any value handle any kind of problem
 $\hookrightarrow$ dynamically changing

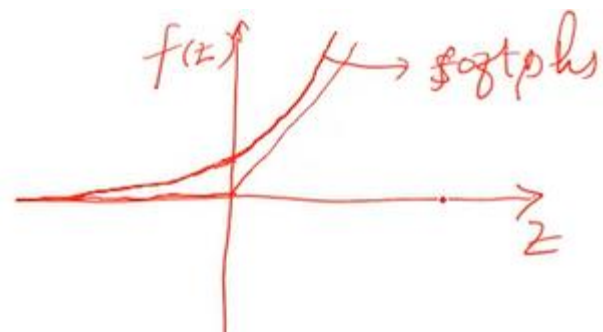
Swish (self-gated)

$$f(z) = z * \text{sigmoid}(z)$$



Softplus:

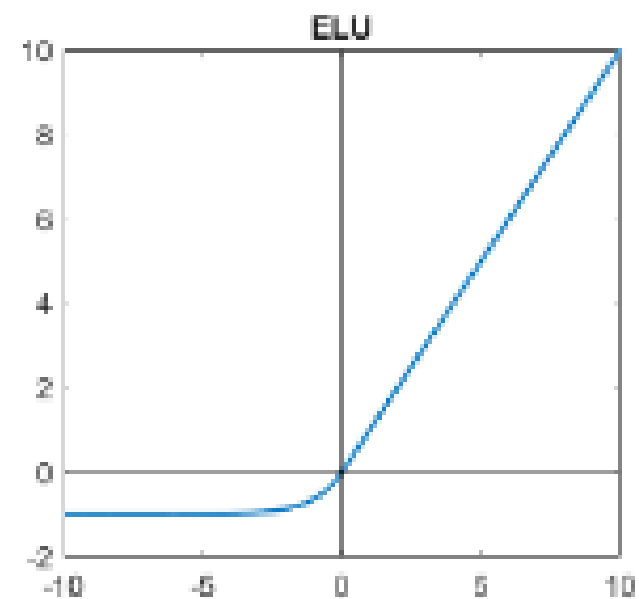
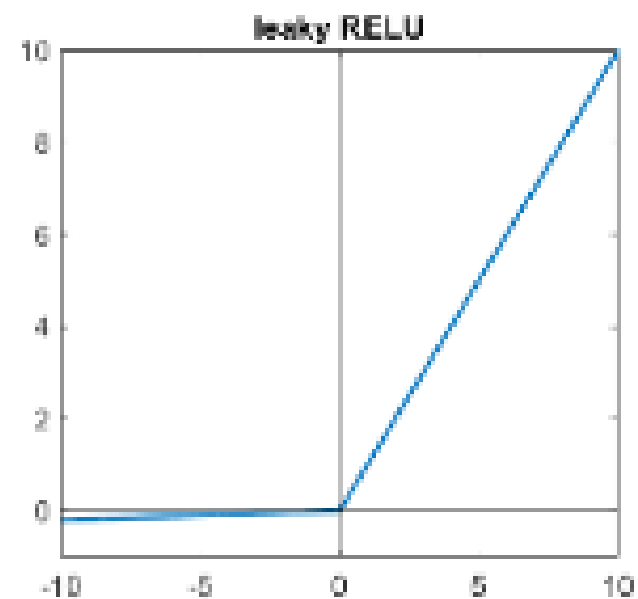
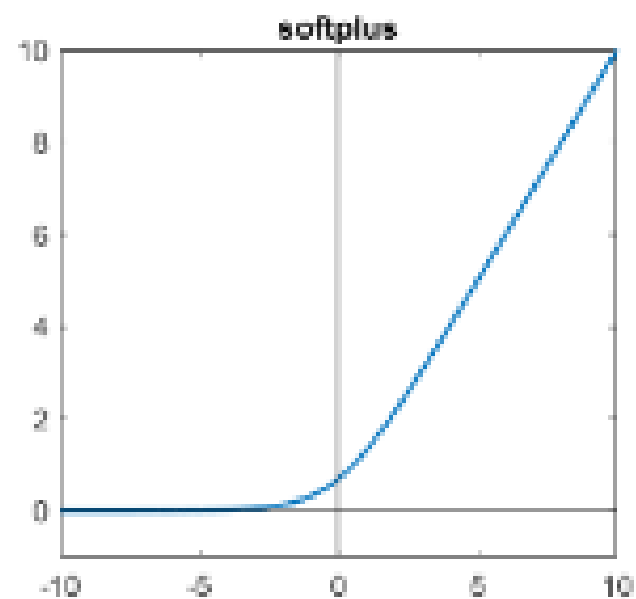
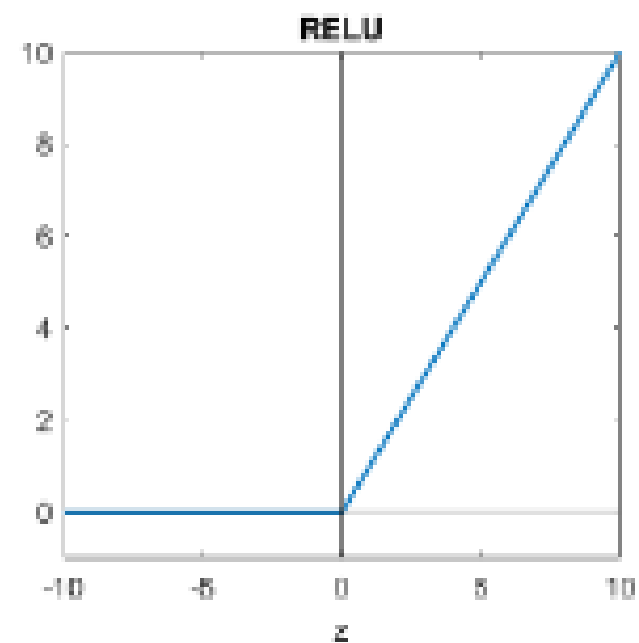
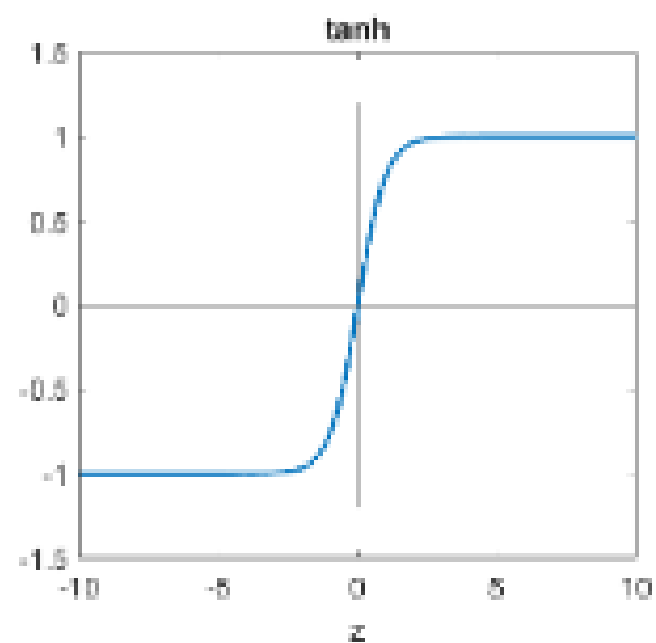
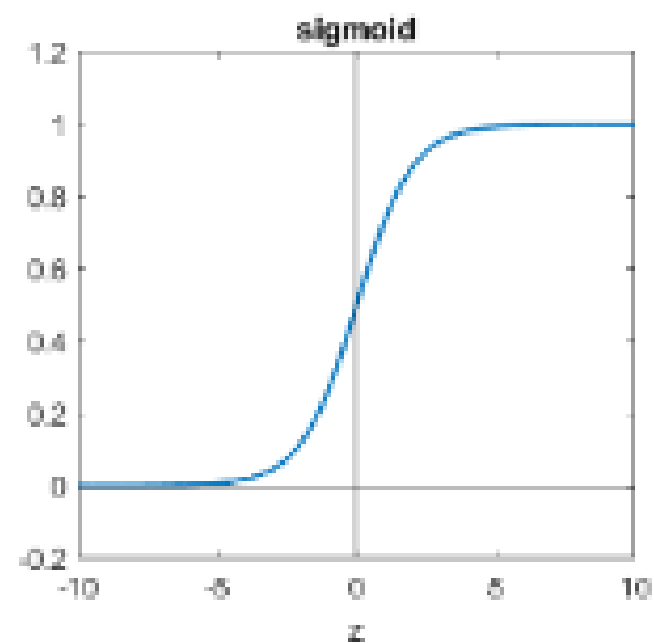
$$f(z) = \ln(1 + e^z)$$



Softmax Activation

$$f(z) = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}}$$

where $j = 1, 2, \dots, K$



Types of data involved

- Training data**
- Validation data**
- Testing data**

Types of learning

- Supervised learning**
- Unsupervised learning**
- Reinforcement learning**
- Semi-supervised learning**

-Stages of Supervised learning

-Error

-Epoch

-Learning Rate

-Stopping condition

-Back propagation

2.6 Types of Data Involved

We know that ANNs need a good amount of data to train themselves. More the data, the better the system's performance.

The data in this approach are generally divided into three sets, each of which is discussed below.

The distribution of data among these sets is the work of the designer and largely depends on the quality and quantity of the data available in the historical database.

- **Training data:** These data are used by the training algorithm to set the ANN's parameters, weights, and biases. Training data make up the largest set of data, comprising almost 80 percent of the data.
- **Validation data:** These data are used for validation purposes during the training period.

At any point during training, we measure the error on both the training data and the validation data.

2.6 Types of Data Involved (Contd..)

If the error in training data is decreasing but on validation data is increasing, we stop the training. The size of this data set is about 10 percent of the entire data set.

- **Testing data:** This data set is used when the final ANN is ready. Testing data, which are completely unknown, are used to test the system's actual performance. If the performance is poor, we may have to reconfigure the ANN. The testing data set consists of about 10 percent of the data.

2.7 Training

ANNs are able to learn the data presented in a process known as *training*.

How to train ANN so that it can adjust itself in such a way that it gives a correct answer whenever a new data is again given to it.

The training algorithm tries to adjust the various weights (and biases) and set them to a value such that the ANN performs better at the applied input.

Thus, the entire training process is a means of finding the right combination of weights and biases for which the ANN performs at its best.

This performance and training depends on the number of hidden layers, as well as on the neurons in these hidden layers.

Changing the architecture cannot be performed during training.

The training algorithm does not play with the network architecture.

2.7 Training (Contd..)

Hence ANN designers try various combinations and numbers of hidden layers and hidden layer neurons to train the network repeatedly in order to get the configuration for the best performance.

Because this entire activity is manual, it may take a long time and a great deal of experience before a designer is able to select a final design.

A good ANN architecture gives the best performance in the least number of layers and the least number of neurons.

2.7 Training (Contd..)

Let us discuss the important terms used in the training process.

2.7.1 Types of Learning

This learning may be broadly classified into three categories.

- **i) Supervised learning:** In this kind of learning, both the inputs and the outputs are well determined and supplied to the training algorithm.

Hence whenever an input is applied, we can calculate the error.

We try to adjust the weights in such a way that this error is reduced.

Supervised learning takes its analogy from the way a person might learn by being assisted by someone who keeps correcting the mistakes.

As a result, the mistakes are reduced to an appreciable extent.

Supervised learning is used for most real life applications.

2.7 Training (Contd..)

2.7.1 Types of Learning (Contd..)

i)Supervised learning (Contd..): The basic block diagram of this learning is given in Figure 2.13.

The training method, and the algorithm that we present below, is a supervised learning technique.

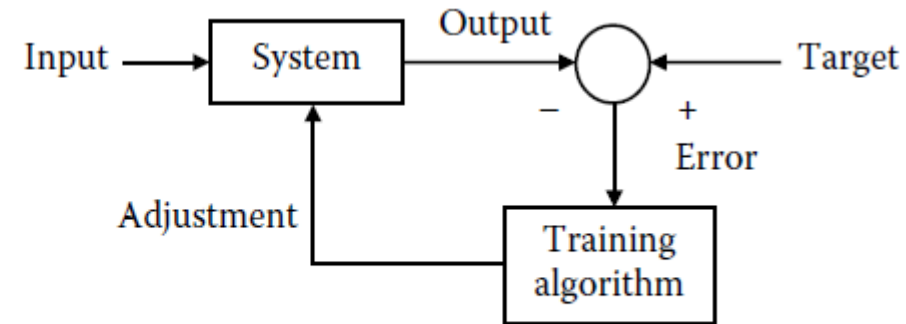


FIGURE 2.13 The basic block diagram of supervised learning.

2.7 Training (Contd..)

2.7.1 Types of Learning (Contd..)

- **Unsupervised learning:** In this type of learning, the target outputs are unknown. The inputs are applied, and the system is adjusted based on these inputs only. Either the supporting weights of the problem are added. The system changes according to the inputs. The result is that the system gets ready to give the correct answer whenever the same input is applied. The unsupervised learning algorithm only needs to calculate the direction of improvement of weight given by the output. The basic block diagram for unsupervised learning is given in Figure 2.14.

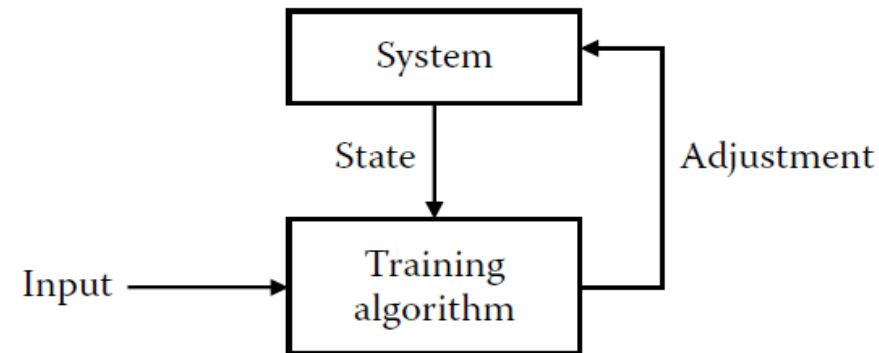
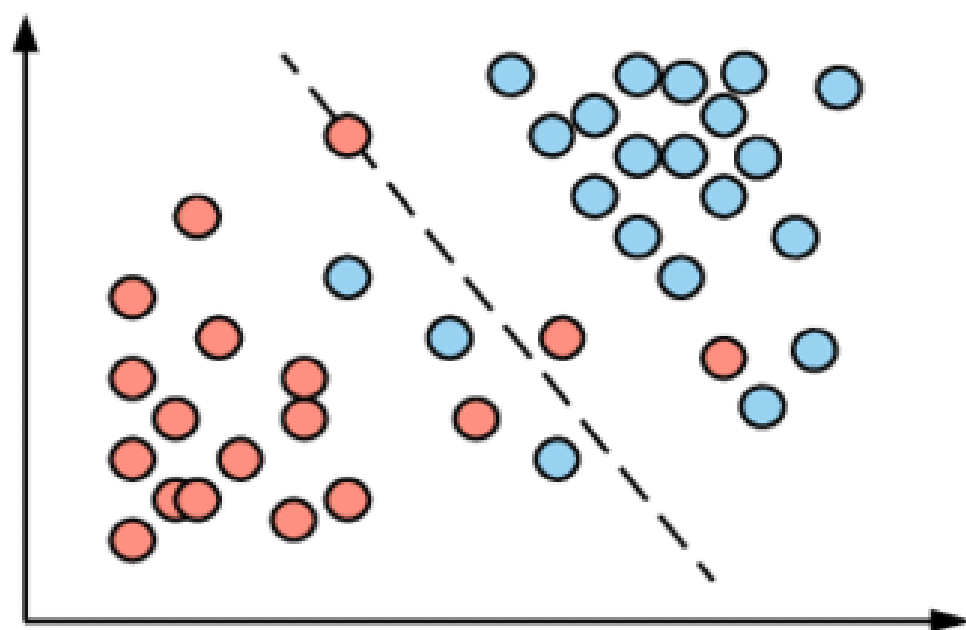
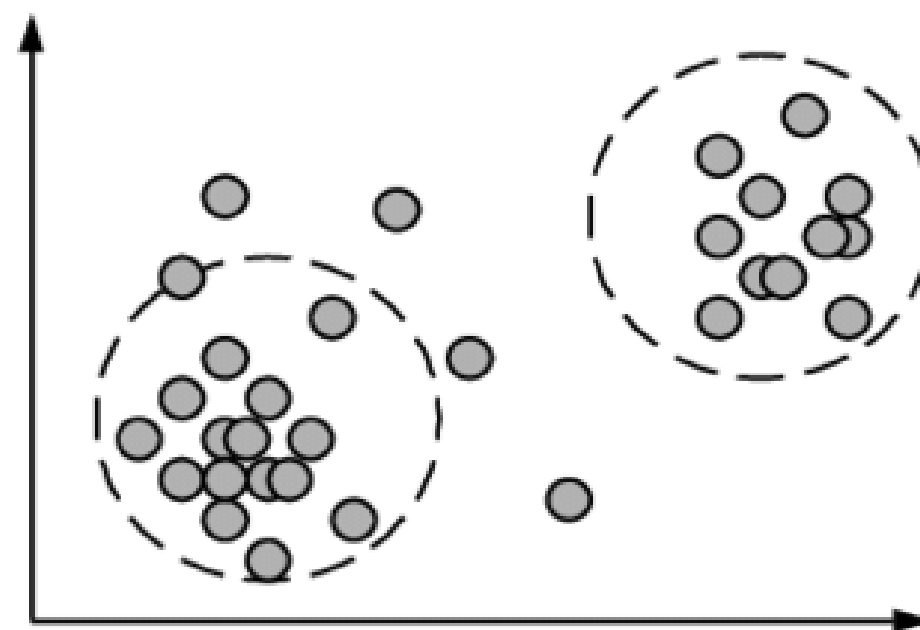


FIGURE 2.14 The basic block diagram of unsupervised learning.



Supervised learning



Unsupervised learning

2.7 Training (Contd..)

2.7.1 Types of Learning (Contd..)

- **Reinforcement learning:** As its name implies, this type of learning is based on the reinforcement process.

In this system, the input is applied. Based on the output, the system either gives some reward to the network or punishes the network.

If the answer was in the correct direction, the result is a reward, and vice versa.

In this learning technique, the system tries to maximize the rewards and minimize the punishment. The basic block diagram is given in Figure 2.15.

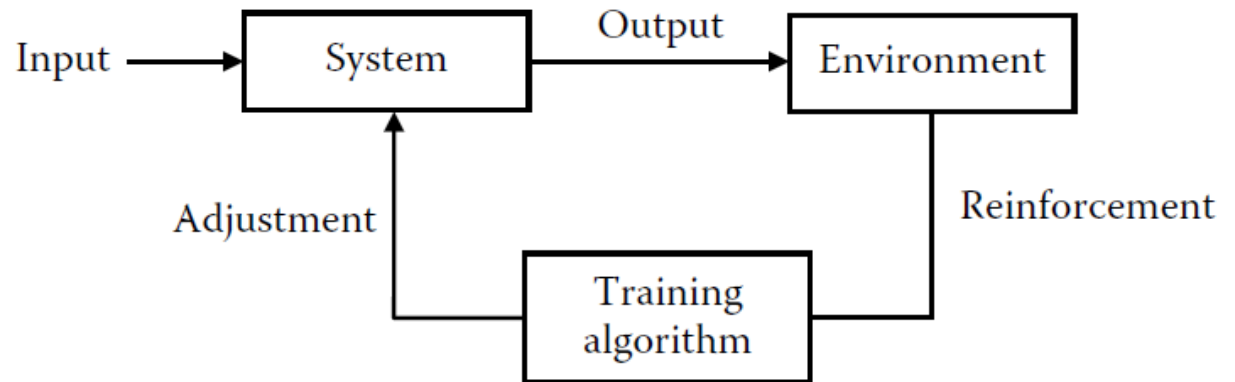


FIGURE 2.15 The basic block diagram of reinforcement learning.

Training (Contd..)

1) Types of Learning (Contd..)

iv) Semi-supervised learning:

Semi-supervised learning is an approach to machine learning that combines a small amount of labeled data with a large amount of unlabeled data during training.

Semi-supervised learning falls between unsupervised learning (with no labeled training data) and supervised learning (with only labeled training data).

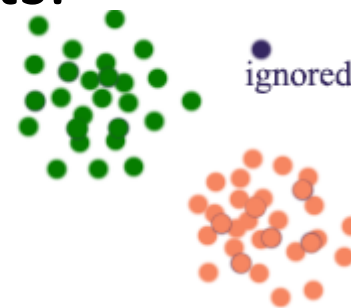
Supervised learning aims to learn a function that, given a sample of data and desired outputs, approximates a function that maps inputs to outputs. Semi-supervised learning aims to label unlabeled data points using knowledge learned from a small number of labeled data points.



Build a model with
labeled data



Place the un-labeled
data with the model



Use the model to label
the un-labeled data



Fit the model again
with the combined data

2.7 Training (Contd..)

2.7.2 The Stages of Supervised Learning

To train the ANN, we need to apply the input, get the output, and accordingly adjust the weights and biases.

ANNs are used for training as well as for testing.

Training deals with giving inputs to the network, noting its errors, and making related modifications. This involves a **forward propagation** step in which the input is applied and the output is retrieved and a **back propagation** step in which the error is calculated and the relevant changes are made.

In testing, however, there is only a forward path.

2.7 Training (Contd..)

2.7.2 The Stages of Supervised Learning (Contd..)

The application of inputs and the calculation of outputs both require the flow of information from one neuron to another until the final output neuron calculates the final answer.

This flow of information is forward, from the input to the output, and it happens layer by layer. This step in the training process is called the *feed-forward step*.

Once the output to the applied input is known, the error can be calculated.

The task now is to adjust the weights according to the error.

This error is propagated backward from the output layer to the input layer.

In this manner, all the weights and biases are adjusted, starting from the output neurons. The general diagram of the two stages is given by Figure 2.16.

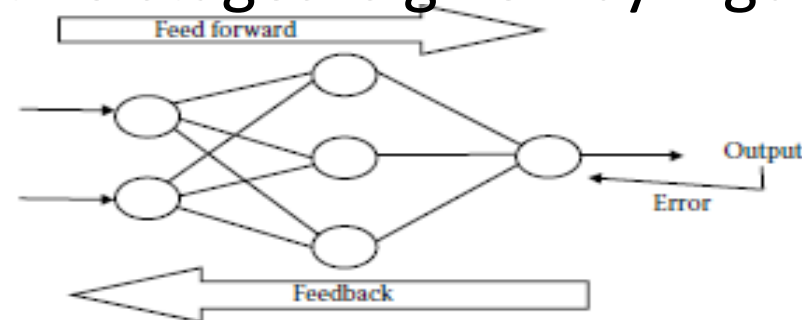


FIGURE 2.16 The feed forward and the feedback stages in the artificial neural network.

2.7 Training (Contd..)

2.7.3 Error Function

Error, a measure of the ANN's performance, helps us in analyzing the ANN.

A good ANN has minimum error.

A good training algorithm moves the ANN from high error configuration to a lower error configuration.

The objective is to minimize the total error associated.

An error in ANN is measured by using error measurement functions.

These error functions represent the net error from all the inputs that are in the training or validating data set.

Various types of error functions may be used, depending on the problem and choice of the designer.

A few of the major, commonly used error functions are :

- **Mean absolute error**
- **Mean squared error with regression**
- **Mean squared error**
- **Sum squared error**

2.7.4 Epoch

ANN training is done in a batch-processing mode, in which a batch consists of all the training data set cases.

This complete pack is presented to the system for training.

After the system finishes processing all these inputs, the same batch is applied again for another iteration of updating the weights and biases.

We continue in this manner, presenting the system with the same pack of inputs and outputs.

A single iteration of applying all the inputs, or the input package, is known a *epoch*.

At every step of this process, the ANN learns to some extent.

As a result of applying the same inputs multiple times, the system is able to train itself with permissible error.

After multiple epochs, the system is said to be fully trained. The number of epochs required for the system to train itself varies according to the problem and the training data.

2.7 Training (Contd..)

2.7.5 Learning Rate

The learning rate is the rate by which the system learns the data.

A larger learning rate means that a system would be able to learn the data very quickly, but it gives larger total errors.

On the other hand, whenever the learning rate is low, the system would learn very slowly, takes a lot of time to train, but the total error of the system is low.

The higher learning rates force an ANN would jump from a point closer to global minima and at the higher rate, it may not be able to jump to this where a minimum lies.

ANN would thus keep oscillating b/n points in an attempt to reach the minima.

If the rate are lower, however, the ANN may take a while to reach the minima, but it would certainly reach it eventually.

2.7 Training (Contd..)

2.7.6 Variable Learning Rate

In the previous section, we saw that the learning rate must be chosen correctly.

A too high or too low value will not perform well.

Each value of learning rate has own problems and limitations.

One solution for choosing the correct learning rate is to keep the learning rate as a variable.

A variable learning rate allows a system to be more flexible. The learning rate at the start is kept high.

As the training continues, however, the learning rate is gradually reduced.

2.7 Training (Contd..)

2.7.8 Stopping Condition

The training algorithm may train itself infinitely, trying to minimize the error more and more. In any real life problem, however, we have to establish when the algorithm should stop training the ANN. The algorithm stops according to the stopping condition.

Normally one or more of the following criteria are used as stopping conditions:

- **Time:** The algorithm may be stopped when the time taken to execute exceeds the threshold.
- **Epoch:** The algorithm has a specified maximum number of epochs. Upon exceeding this number, the algorithm may be stopped.
- **Goal:** The algorithm may be stopped if the error measured by the system reduces to more than a specific value. It may not be useful to continue training after this point.
- **Validating data:** If the error on validation data starts increasing, even if there is a decrease in the testing data, it would be better to stop further training.

2.7 Training (Contd..)

2.7.8 Stopping Condition (Contd..)

- **Gradient:** Gradient refers to the improvement of the performance or the lowering of the error in between epochs.

If the algorithm fails to improve the performance or lower the error appreciably between epochs, the training may be stopped.

Here it is assumed that further training would not result in much of a difference in performance.

2.7 Training (Contd..)

2.7.9 Back Propagation Algorithm

One of the algorithms used to train the ANN.

This algorithm, called the back propagation algorithm (BPA), is a supervised learning algorithm that calculates the error and then back propagates the error to previous layers.

These previous layers then adjust their weights and biases accordingly, and the configuration is thus changed.

2.7 Training (Contd..)

2.7.9 Back Propagation Algorithm (Contd..)

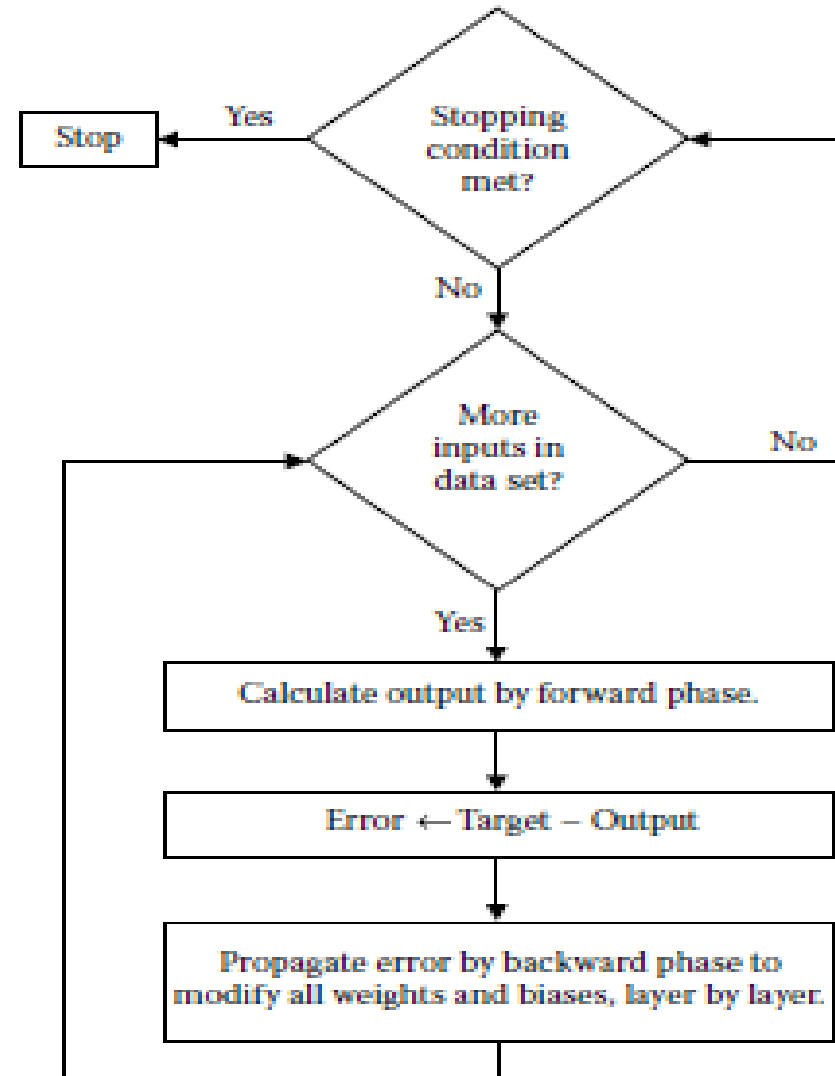


FIGURE 2.19 The flow diagram of the back propagation algorithm.